

AUTOMATED WASTE SEGREGATION SYSTEM USING YOLO AND FLASK

A PROJECT REPORT

Submitted by

KISHORE KHANNAN H [RA2211026010409]
KOUSHIK VISHAL S [RA2211026010384]

Under the Guidance of

Dr. Jackulin Mahariba A

Assistant Professor, CINTEL

in partial fulfillment of the requirements for the degree of

BACHELOR OF TECHNOLOGY
in
COMPUTER SCIENCE ENGINEERING
WITH SPECIALIZATION IN ARTIFICIAL INTELLIGENCE
AND MACHINE LEARNING



DEPARTMENT OF COMPUTATIONAL INTELLIGENCE
COLLEGE OF ENGINEERING AND TECHNOLOGY
SRM INSTITUTE OF SCIENCE AND TECHNOLOGY
KATTANKULATHUR- 603 203

MAY 2026



Department of CINTEL
SRM Institute of Science & Technology
Own Work Declaration Form

Degree/ Course : B.Tech CSE AIML
Student Name : KISHORE KHANNAN H / KOUSHIK VISHAL S
Registration Number : RA2211026010409 / RA2211026010384
Title of Work : AUTOMATED WASTE SEGREGATION SYSTEM USING YOLO AND FLASK

We hereby certify that this assessment compiles with the University's Rules and Regulations relating to academic misconduct and plagiarism, as listed in the University Website, Regulations, and the Education Committee guidelines.

We confirm that all the work contained in this assessment is my / our own except where indicated, and that We have met the following conditions:

- Clearly referenced / listed all sources as appropriate
- Referenced and put in inverted commas all quoted text (from books, web, etc)
- Given the sources of all pictures, data etc. that are not my own
- Not made any use of the report(s) or essay(s) of any other student(s) either past or present
- Acknowledged in appropriate places any help that I have received from others (e.g. fellow students, technicians, statisticians, external sources)
- Compiled with any other plagiarism criteria specified in the Course handbook /University website

I understand that any false claim for this work will be penalized in accordance with the University policies and regulations.

DECLARATION:

I am aware of and understand the University's policy on Academic misconduct and plagiarism and I certify that this assessment is our own work, except where indicated by referring, and that I have followed the good academic practices noted above.



SRM INSTITUTE OF SCIENCE AND TECHNOLOGY KATTANKULATHUR – 603 203

BONAFIDE CERTIFICATE

Certified that 21CSP401L – Major Project [21CSP403L- Internship] report titled “**AUTOMATED WASTE SEGREGATION SYSTEM USING YOLO AND FLASK**” is the bonafide work of “ **KISHORE KHANNAN H [RA2211026010409] , KOUSHIK VISHAL S [RA2211026010384]**”, who carried out the project work[internship] under my supervision. Certified further, that to the best of my knowledge the work reported herein does not form any other project report or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

SIGNATURE

Dr. JACKULIN MAHARIBA A

**SUPERIOR
ASSISTANT PROFESSOR
DEPARTMENT OF
COMPUTATIONAL INTELLIGENCE
SRM INSTITUTE OF SCIENCE AND
TECHNOLOGY**

SIGNATURE

Dr. R. ANNIE UTHRA

**HEAD OF THE DEPARTMENT
PROFESSOR
DEPARTMENT OF
COMPUTATIONAL INTELLIGENCE
SRM INSTITUTE OF SCIENCE AND
TECHNOLOGY**

EXAMINER 1

EXAMINER 2

ACKNOWLEDGEMENTS

We express our humble gratitude to **Dr. C. Muthamizhchelvan**, Vice-Chancellor, SRM Institute of Science and Technology, for the facilities extended for the project work and his continued support.

We extend our sincere thanks to **Dr. Leenus Jesu Martin M**, Dean-CET, SRM Institute of Science and Technology, for his invaluable support.

We wish to thank **Dr. Revathi Venkataraman**, Professor and Chairperson, School of Computing, SRM Institute of Science and Technology, for her support throughout the project work.

We encompass our sincere thanks to, **Dr. M. Pushpalatha**, Professor and Associate Chairperson - CS, School of Computing and **Dr. C Lakshmi**, Professor and Associate Chairperson -AI, School of Computing, SRM Institute of Science and Technology, for their invaluable support.

We are incredibly grateful to our Head of the Department, **Dr .R. Annie Uthra**, HOD & Computational Intelligence , SRM Institute of Science and Technology, for her suggestions and encouragement at all the stages of the project work.

We want to convey our thanks to our Project Coordinators, Panel Head, and Panel Members Department of Computational Intelligence, SRM Institute of Science and Technology, for their inputs during the project reviews and support.

We register our immeasurable thanks to our Faculty Advisor ,**Dr. Varalakshmi I and Dr Salomi Samsudeen M** , Department of CINTEL, SRM Institute of Science and Technology, for leading and helping us to complete our course.

Our inexpressible respect and thanks to our guide, **Dr Jackulin Mahariba A**, Department of Computational Intelligence, SRM Institute of Science and Technology, for providing us with an opportunity to pursue our project under her mentorship. She provided us with the freedom and support to explore the research topics of our interest. Her passion for solving problems and making a difference in the world has always been inspiring.

We sincerely thank all the staff members of Computational Intelligence, School of Computing, SRM Institute of Science and Technology, for their help during our project. Finally, we would like to thank our parents, family members, and friends for their unconditional love, constant support and encouragement

Kishore Khannan H

Koushik Vishal S

ABSTRACT

In recent times, handling waste effectively has become a major issue due to the continuous increase in garbage generation from both urban and rural areas. One of the key problems in waste management is proper segregation, which is still mostly done manually in many places. This process takes time, is not always accurate, and can also be unsafe for workers who are directly exposed to waste materials. Because of these challenges, there is a need for a smarter and more reliable solution. This project proposes an automated waste classification system that uses deep learning to identify different types of waste. The system is designed to recognize materials such as plastic, paper, and metal by analyzing images. A YOLO-based object detection model is used because it can perform detection quickly and efficiently in real-time situations. To train the model, a dataset of waste images was collected, cleaned, and properly labeled. The performance of the model was evaluated using parameters such as precision, recall, and mean average precision. Over time, the results showed improvement, and the model was able to make better predictions with reduced errors. After training, the model was integrated into a web application so that users can easily interact with it. The application provides options for live detection as well as image upload for testing. It displays the predicted category along with a confidence score, which helps users understand how accurate the result is. A history feature is also included to store previous outputs. The interface is kept simple so that even users without technical knowledge can use it without difficulty. The system was tested under different conditions. It performed well when images were clear and had proper lighting. However, the accuracy slightly decreased in low-light environments or when the background was complex. These results show that there is still scope for improvement. Overall, this project shows how deep learning can be applied in waste management to reduce manual effort and improve efficiency. It also creates opportunities for future developments such as mobile applications, IoT-based smart bins, and real-time large- scale usage.

TABLE OF CONTENTS

ABSTRACT		iv
LIST OF FIGURES		vii
LIST OF TABLES		viii
ABBREVIATIONS		ix
CHAPTER	TITLE	PAGE
NO		NO
1	INTRODUCTION	1
1.1	Introduction to Project	1
1.2	Problem Statement	2
1.3	Motivation	2
1.4	Sustainable Development Goal of the Project	3
2	LITERATURE SURVEY	4
2.1	Overview of the Research Area	8
2.2	Existing Models and Frameworks	9
2.3	Limitations Identified from Literature Survey	9
2.4	Research Objectives	10
2.5	Product Backlog	11
2.6	Plan of Action	12
3	SPRINT PLANNING AND EXECTION METHODOLOGY	14
3.1	DATASET PREPARATION PHASE	14
3.1.1	Objectives with user stories of Sprint I	14
3.1.2	Functional Document	15
3.1.3	Architecture Document	17
3.1.4	Outcome of objectives	19
3.1.5	Sprint Retrospective	21
3.2	MODEL DEVELOPMENT PHASE	23
3.2.1	Objectives with user stories of Sprint II	23
3.2.2	Functional Document	24
3.2.3	Architecture Document	26
3.2.4	Outcome of objectives	28

3.2.5	Sprint Retrospective	29
3.3	APPLICATION INTEGRATION PHASE	30
3.3.1	Objectives with user stories of Sprint III	30
3.3.2	Functional Document	30
3.3.3	Architecture Document	33
3.3.4	Outcome of objectives	36
3.3.5	Sprint Retrospective	39
4	RESULTS AND DISCUSSIONS	40
4.1	Project Outcomes	40
5	CONCLUSION AND FUTURE ENHANCEMENT	48
	REFERENCES	51
	APPENDIX	
A	CODING	52
B	PATENT DISCLOSURE FORM	68
C	PLAGIARISM REPORT	70

LIST OF FIGURES

FIGURE	TITLE	PAGE NO
2.1	Product Backlog	12
3.1	System Architecture	18
3.2	Waste Dataset Categories	20
3.3	Augmented Waste Image Variations	20
3.4	Annotated Waste Images	21
3.5	Sprint Retrospective	22
3.6	System Architecture	27
3.7	High Confidence Detection	28
3.8	Low Confidence Live Detection	29
3.9	Sprint Retrospective	29
3.10	System Architecture	34
3.11	Main Dashboard Screen	36
3.12	Low Accuracy Live Output	37
3.13	Live Waste Detection	37
3.14	Saved Detection History	38
3.15	Sprint Retrospective	39
4.1	Optimal Confidence Curve	41
4.2	Precision vs Confidence	42
4.3	Confusion Matrix Counts	44
4.4	Normalized Confusion Matrix	44
4.5	System Dashboard Interface	45
4.6	Real-Time Detection Output	46
4.7	Stored Detection Results	47

LIST OF TABLES

TABLE	TITLE	PAGE NO.
2.1	Plan of Action	11
3.1	Objectives of User stories.	14
3.2	Authorization Matrix	17
3.3	Services and Responsibilities	17
3.4	Data Exchange Contract	19
3.5	Objectives of User Stories	23
3.6	Authorization Matrix	25
3.7	Services and Responsibilities	26
3.8	Data Exchange Contract	28
3.9	Objectives of User stories.	30
3.10	Authorization Matrix	32
3.11	Services and Responsibilities	33
3.12	Data Exchange Table	35
4.1	Performance Evaluation	41
4.2	Comparison of images taken at different working condition	43

ABBREVIATIONS

AIML	Artificial Intelligence Machine learning
YOLO	You Look Only Once
CNN	Convolutional Neural Network
API	Application Programming Interface
UI	User Interface
DB	Data Base
HTML	Hyper Text Markup Language
HTTP	Hyper Text Transfer Protocol
JS	Javascript
IDE	Integrated Development Environment

CHAPTER 1

INTRODUCTION

1.1 Introduction to Project

Waste management has become a growing concern in recent years due to rapid urbanization, industrial development, and increasing population. Every day, a large quantity of waste is generated from households, industries, and commercial areas. If this waste is not handled properly, it can lead to environmental pollution and health-related issues. One of the most important steps in waste management is segregation, where waste is separated into categories such as biodegradable, recyclable, and non-recyclable materials. However, in many places, waste is not segregated at the source. Different types of waste are often mixed together, which makes recycling more difficult and less effective. Most existing systems depend on manual sorting, which requires more time and effort. In addition, manual methods are not always accurate and may expose workers to harmful substances present in the waste. To overcome these problems, there is a need for an automated and efficient system that can perform waste segregation with minimal human involvement. This project focuses on developing an automated waste segregation system using deep learning techniques. In particular, the YOLO (You Only Look Once) algorithm is used for detecting and classifying waste materials in real time. The system is designed to identify different types of waste such as plastic, paper, metal, and organic materials based on visual input. Along with the detection model, a web-based interface is developed using Flask to make the system easy to use. Users can view results, monitor detection outputs, and interact with the system through this interface. By combining machine learning with web technologies, the proposed system aims to improve the speed and accuracy of waste segregation. It also reduces the dependency on manual labour and provides a safer and more efficient solution. Such a system can be useful in areas like smart cities, recycling units, and modern waste management systems.

1.2 Problem Statement

Waste segregation is a key part of effective waste management, yet it is often not carried out correctly in real-world situations. In many cases, people dispose of different types of waste together instead of separating them at the source. This leads to mixing of recyclable, non-recyclable, and organic waste, which makes the recycling process more complicated and less efficient. Manual waste sorting methods are still widely used, but they come with several drawbacks. These methods require a large amount of human effort and are time-consuming, especially when dealing with high volumes of waste. In addition, the accuracy of manual segregation is not always reliable, as it depends on human judgment, which can vary. Workers involved in waste handling are also exposed to unhealthy and sometimes hazardous conditions. While automated waste segregation systems are available, many of them are either costly or not efficient enough for continuous real-time use. Some systems also struggle to identify waste correctly when there are changes in lighting, object shape, or background conditions. Because of these limitations, their performance may not be consistent in practical environments. Therefore, there is a need for a system that can automatically detect and classify different types of waste with better accuracy and minimal human involvement. This project aims to address these challenges by using a YOLO-based object detection model along with a Flask-based interface. The goal is to develop a system that improves efficiency, reduces human effort, and provides a practical solution for modern waste management.

1.3 Motivation

The idea for this project comes from the growing challenges seen in waste management across many regions. Problems such as overflowing garbage bins, environmental pollution, and ineffective recycling systems are still common, which indicates that existing methods are not sufficient to manage waste efficiently. These issues highlight the need for improved and more practical solutions. Another factor that influenced this work is the rapid progress in artificial intelligence and machine learning. These technologies now allow many tasks to be handled automatically, including identifying and classifying objects from images. Among different approaches, the YOLO algorithm is widely used for its ability to perform

quick and reliable detection, which makes it suitable for real-time applications such as waste classification. Reducing the dependency on manual labour is also an important motivation behind this project. Traditional waste sorting methods require continuous human effort and can expose workers to unsafe and unhygienic conditions. An automated system can help minimize these risks while also improving the overall efficiency of the process. Additionally, this project provides an opportunity to make use of concepts from machine learning, computer vision, and web development in a practical setting. It shows how technical knowledge can be applied to address real-world challenges and contribute towards environmental improvement. Overall, the project focuses on developing a system that supports cleaner and more sustainable waste management practices.

1.4 Sustainable Development Goal of the Project

This project is aligned with a number of Sustainable Development Goals (SDGs) defined by the United Nations, which focus on improving environmental conditions and promoting sustainable growth.

- **Responsible Consumption and Production (SDG 12):**

The system supports proper separation of waste, which makes recycling processes more effective and helps decrease the volume of waste that ends up in landfill sites.

- **Sustainable Cities and Communities (SDG 11):**

By improving the way waste is managed, the project contributes to maintaining cleaner surroundings and supports the development of more sustainable urban areas.

- **Climate Action (SDG 13):**

Better handling of waste can help limit environmental pollution and reduce the release of harmful gases that are usually produced when waste is not managed properly.

- **Good Health and Well-being (SDG 3):**

Reducing direct human interaction with waste can lower health risks and create safer conditions for both workers and the public.

CHAPTER 2

LITERATURE SURVEY

Waste segregation research [1] indicates inefficiency and utilization of manual sorting operations by recycling facilities as an issue. Convolutional neural network architectures such as ResNet-18 with trade-off between performance of classification and computation cost towards real-time computing are the areas of current studies in machine learning methodologies. ResNet-18 has been successfully implemented in recycling process application for the separation of wastes into categories with increased sorting effectiveness and capability . Hardware system integration allowing automatic physical waste product separation is also facilitated with output of CNN-based classification with limited human participation. The method is low-cost and scalable, and thus a suitable solution to the recycling issue currently experienced.

Waste management literature is extremely eager to point out the severe necessity of good rubbish segregation as a step to reduce the environmental issue generated by the aggressive industrialization and urbanization. While they are in intense use, manual sorting techniques are vulnerable to errors and unsafe for the human resource that will be handling toxic chemicals. In an effort to mechanize waste sorting, scientists have pursued various technologies ranging from deep learning and image processing algorithms. Of the notable innovations is the utilization of CNN architecture such as ResNet and variants of YOLO [2] that enhance classification accuracy and support real-time waste type detection. These technologies, employing smart waste management protocols, seek to render processes environmentally friendly, decrease human contact, and accelerate recycling.

Current waste management is strained due to growing urbanization, and new solutions are in the works. This paper proposes Enviro Tech, an Internet of Things-based autonomous waste segregation system on a Raspberry Pi 4. OpenCV and the MobileNetV3 [3] algorithm are utilized for real-time garbage detection and garbage classification as biodegradable and non-biodegradable garbage with a success rate of 96.32%. Garbage level detecting sensors and a deodorizer for biodegradable garbage are also proposed, with user experience improvement. A fire detecting sensor is proposed for non-biodegradable garbage, with an early warning system. Enviro Tech is beneficial in favoring smart waste management systems by way of scalability and flexibility and favoring sustainable cities. Current research has been on waste management systems, e.g., real-time bin status and collection time on smart solid waste bins. Enhanced features like odor control and fire detection are missing in current research. The lacunas are addressed by this paper with an integrated solution proposed. The system is on a Raspberry Pi 4 microprocessor with real-time edge processing capability support.

Waste management is obstructed by the lack of diversified data, and hence AI-based systems perform poorly. Here, a real-time detection and identification of Bangladesh-specific solid waste images is recommended through a deep-learning approach. Researchers compiled the dataset from public sources and combined them with additional annotated images with the intention to provide more precision. The system identifies 12 waste types, i.e., plastic, metal, paper, and e-waste, with 73% classification accuracy and F1 score 0.729. [4] Traditional waste management systems are labor and time consuming since they require manual labor. The AI-based waste identification system will be efficient and save the environment as it will automatically detect waste. The data utilized is over 13,000 images of waste. hence data is diverse and well-distributed among categories. Four object detection models were utilized, which include ResNet50, SSD Resnet101, YOLOv5, and a custom R-CNN. The custom R-CNN model was adjusted to manage more than one annotation in one image better. This research fills the research gap in waste management by viewing Bangladesh as a unique case. This research opens new ways for data science to be applied to waste management.

One of the main difficulties in waste management is accurately identifying and separating different types of waste materials. With the rapid increase in population and consumption, the amount of municipal waste is expected to rise significantly in the coming years, creating additional pressure on existing systems. The proposed [5] model focuses on improving detection performance by using advanced object detection techniques. Experimental results indicate that the system is able to achieve strong performance in terms of classification accuracy and consistency. Evaluation metrics such as precision and recall show that the model performs reliably in identifying different categories of waste under various conditions.

This study [6] presents a waste detection system designed for use in recycling environments using deep learning techniques. The model is developed using data collected from industrial settings, particularly from conveyor belt operations where waste materials are processed continuously. The approach used in this system follows a modern detection strategy that differs from traditional methods which rely on predefined anchor boxes. Instead, it focuses on a more direct prediction mechanism that improves detection efficiency and simplifies the overall process. The trained model is capable of identifying and classifying multiple types of waste materials across a wide range of categories. This makes the system suitable for real-time applications in large-scale recycling units, where speed and accuracy are important for effective waste sorting.

Accurate classification forms the basis of efficient garbage disposal, but manual sorting is no longer effective. The article explores deep learning for recycling product classification following Saudi Arabian sustainability. The work compares several models of CNNs (DenseNet169, MobileNetV2, ResNet50V2) in classifying rubbish. ResNet50V2 had maximum accuracy (98.95%), when compared to CNN model (88.52%). [7] This is an automated procedure that improves waste management with reduced labor and human errors. Previous studies include experiments with deep networks and CNN along with improved image processing. Research has also been carried out on dual-branch CNN and waste detection methods using Efficient Net. All these methods are founded on the promise

of deep learning to waste management. Recycling is improved with such studies towards a sustainable future. It aligns with Saudi Vision 2030 towards a sustainable kingdom. Don't add anything just give in format to paste in report

This study describes an automated system that uses a YOLO-based model to identify and sort different types of waste materials in real time. The model analyzes input images and categorizes waste with consistent performance and fast processing speed. The system is built to handle different environmental conditions, such as variations in lighting and changes in object positioning[8]. This makes it more adaptable to real-world situations where conditions are not always controlled. By reducing the need for manual involvement, the approach helps improve sorting efficiency and minimizes human effort. The results suggest that such systems can be useful in practical applications, including recycling facilities and smart waste management setups.

Waste classification involves dealing with different types of objects that often appear in complex and unstructured environments. This study discusses various computer vision methods used for detecting and classifying garbage, including convolutional neural network (CNN) models and different versions of YOLO [9]. The comparison focuses on key aspects such as processing speed, detection performance, and the ability to identify multiple objects within a single frame. It also highlights common difficulties, such as overlapping waste items and imbalance in available training data. From the analysis, it is observed that although many models provide acceptable accuracy, YOLO-based methods are more suitable for real-time applications due to their faster response and efficient processing. Overall, the study gives a clear idea of how different algorithms can be used for waste classification and their effectiveness in practical scenarios.

Automation in waste management requires systems that can handle data efficiently and provide quick results without delays. This study presents a deep learning model based on an advanced version of YOLO, designed for real-time detection and classification of waste materials. The model is trained through multiple iterations and evaluated based on its ability

to correctly recognize different types of objects. The performance shows consistent results, with reasonable accuracy and stable detection speed under various conditions. This approach is suitable for applications that require continuous monitoring, such as conveyor-based waste sorting systems. The study indicates that newer versions of YOLO models can effectively support real-time waste management by improving overall efficiency and reliability.

2.1 Overview of the Research Area

Waste management has gained significant attention in recent years due to growing environmental issues and the need for sustainable solutions. Among the various stages involved, waste segregation is considered one of the most important, as proper separation directly influences the effectiveness of recycling processes. With advancements in technology, researchers are increasingly exploring the use of artificial intelligence and computer vision to automate waste classification. These systems are designed to recognize different categories of waste materials by analyzing visual characteristics such as shape, color, and texture. Such approaches help reduce reliance on manual sorting and improve overall efficiency. Deep learning methods, particularly Convolutional Neural Networks (CNNs), have proven to be effective in handling image-based classification tasks. In addition to this, object detection models like YOLO are commonly used because they can identify multiple objects within a single frame while maintaining fast processing speed. This makes them suitable for real-time applications, including automated segregation systems and smart waste bins. Apart from detection, web-based technologies also play an important role in managing these systems. Frameworks such as Flask are used to build user-friendly interfaces that allow monitoring, storing, and analyzing data efficiently. The integration of AI models with web applications enables better control and accessibility. Overall, the research area is moving towards the development of intelligent and automated waste management systems that can operate effectively in real-world conditions while supporting sustainable practices.

2.2 Existing Models and Frameworks

Over the years, different approaches have been developed to improve waste segregation using various technologies. In the early stages, simple image processing methods were used, where features such as color patterns and edges were analyzed to distinguish between waste types. However, these techniques were not very reliable, especially when objects had similar visual characteristics. With the introduction of machine learning, models like Support Vector Machines and decision tree algorithms were applied to improve classification results. Although these methods performed better than basic image processing, they still depended heavily on manual feature selection, which limited their effectiveness when dealing with complex and large datasets. As deep learning techniques became more popular, models based on neural networks, particularly Convolutional Neural Networks (CNNs), started to show improved performance. These models are capable of learning important features directly from images, which reduces the need for manual intervention and increases accuracy. Many studies have successfully used CNN-based methods for identifying different categories of waste. In recent developments, object detection models such as YOLO, Faster R-CNN, and SSD have been widely explored. Among these, YOLO is often preferred because it processes the entire image in a single pass, allowing faster detection suitable for real-time systems. Some implementations also combine detection models with automated mechanisms to separate waste materials. Additionally, certain systems integrate IoT components, where sensors and smart bins are used to collect data and monitor waste levels. Web frameworks like Django or Flask are often included to create dashboards for tracking system performance and storing data. Although these approaches have shown promising results, there is still room for improvement in terms of scalability, cost, and consistent accuracy in real-world conditions.

2.3 Limitations Identified from Literature Survey

Although several systems have been proposed for automated waste segregation, a number of limitations can still be observed from existing studies. One of the main issues is related to accuracy. In many cases, models struggle when waste objects overlap, are only partially visible, or appear under different lighting conditions. This reduces the reliability of such

systems when applied in real-world environments. Another concern is the speed of operation. Some approaches achieve good accuracy but take more time to process, which makes them less suitable for real-time applications. Systems that depend on multiple stages of processing may introduce delays and affect overall performance. Cost is also an important factor. Certain solutions require advanced hardware or complex configurations, which can increase the overall expense and limit their practical use, especially in large-scale deployments. In addition, a few systems lack user-friendly interfaces, making it difficult to monitor outputs or manage the system effectively. It is also observed that many studies focus mainly on classification and do not provide a complete solution that includes detection, segregation, and monitoring together. These observations indicate the need for a system that balances accuracy, speed, and cost while remaining easy to use. The proposed project attempts to address these gaps by using a YOLO-based detection model along with a Flask-based interface for better usability.

2.4 Research Objectives

The primary aim of this project is to develop an automated waste segregation system by making use of modern technologies and intelligent models.

The key objectives of the project are listed below:

- To design a model capable of identifying and categorizing various types of waste materials, including plastic, paper, metal, and organic waste, with good accuracy.
- To implement a YOLO-based approach for real-time detection so that the system can operate efficiently with faster response time.
- To build a web-based platform using Flask that allows users to monitor outputs and manage system activities easily.
- To decrease the dependence on manual waste handling and reduce the health risks associated with it.
- To enhance the overall effectiveness of waste management and support better recycling practices.

- To develop a solution that is practical, affordable, and suitable for real-world implementation.

2.5 Product Backlog

Table 2.1 Plan of Action

ID	Title	Epic	User Story	Priority	Status	Acceptance Criteria	Functional Requirements	Non-Functional Requirements	Estimated Effort
1	Waste Detection	Real-time Processing	As a user, I want the system to detect waste objects in real time so that segregation can be automated.	High	In Progress	Waste objects are detected with good accuracy using camera input.	YOLO model processes live video/images.	System should be fast and accurate.	12 hours
2	Waste Classification	AI Model	As a user, I want the system to classify waste into categories so that proper segregation can be done.	High	To Do	Waste is classified into categories like plastic, metal, paper, etc.	Model training and prediction module.	High accuracy and reliability required.	14 hours
3	Image Capture	Input Handling	As a user, I want the system to capture images of waste so that detection can be performed.	High	Done	Images are captured clearly from camera.	Camera integration with system.	Should work in different lighting conditions.	6 hours
4	Web Dashboard	Monitoring	As a user, I want a dashboard to view system results so that I can monitor performance.	High	In Progress	Detected results are displayed on web interface.	Django-based frontend and backend.	User-friendly and responsive UI.	16 hours
5	Data Storage	Database	As a user, I want detected data to be stored so that I can analyze it later.	Medium	To Do	Data is stored properly in database.	Database integration (SQLite/MySQL).	Data should be secure and consistent.	10 hours
6	Alert System	Notifications	As a user, I want alerts for system status so that I can take action if needed.	Medium	To Do	Alerts are generated for errors or detection updates.	Notification module.	Timely and reliable alerts.	8 hours
7	System Integration	Deployment	As a developer, I want to integrate all components so that the system works smoothly.	High	In Progress	All modules work together without errors.	Integration of YOLO + Django.	System should be stable and efficient.	18 hours
8	Performance Optimization	System Improvement	As a user, I want the system to run efficiently so that it can handle real-time tasks.	Medium	To Do	System runs without lag.	Optimization of model and backend.	Low latency and high performance.	10 hours

The project backlog outlines key features required to build an effective waste detection and monitoring system. It begins with real-time waste detection using camera input, where the YOLO model processes live images to identify waste with good accuracy. This is followed by waste classification, where detected items are grouped into categories such as plastic, metal, paper, and others to support proper segregation. The system also includes an image capture module to ensure clear input under different lighting conditions. A web-based dashboard is developed using Flask to display results and allow users to monitor system

performance easily. Additionally, detected data is stored securely in a database for future analysis, while a notification system is included to generate alerts for errors or important updates. All components are integrated to work smoothly together, ensuring stable performance. Finally, the system is optimized to handle real-time tasks efficiently with low latency and reliable output.

2.6 plan of action

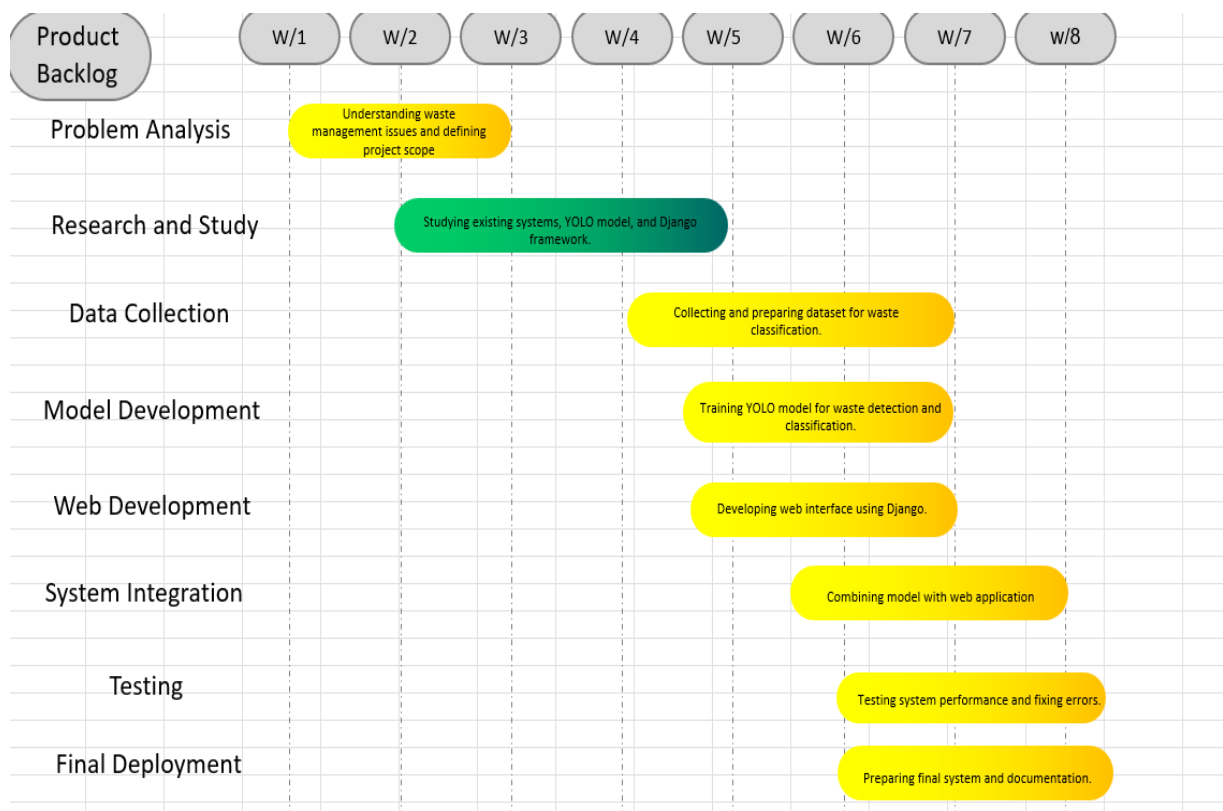


Figure 2.1 Product Backlog

The project was carried out in a structured manner over several weeks, beginning with problem analysis to understand key challenges in waste management and to define the project scope. This was followed by research and study of existing systems, including the YOLO model and Flask framework, to build a strong technical foundation. In the next phase, relevant data was collected and prepared to support accurate waste classification. Model development involved training the YOLO algorithm for effective detection and classification of waste materials.

Parallel to this, a web interface was developed using Flask to enable user interaction. The system integration phase combined the trained model with the web application to ensure smooth functionality. Testing was then conducted to evaluate performance and correct any errors. Finally, the project concluded with deployment, along with proper documentation of the system.

CHAPTER 3

SPRINT PLANNING AND EXECUTION METHODOLOGY

3.1 Dataset Preparation Phase

3.1.1 objective with userstory of sprint 1.

The first sprint was focused on setting up the initial stage of the system. Since the model's performance is closely linked to the quality of data, greater importance was given to collecting relevant samples and preparing the dataset in a structured way. This step ensured that the model would be able to learn patterns that reflect real-world conditions

Table 3.1 Objectives of Userstories

Objective	User Story	Acceptance Criteria
Dataset Collection	As a developer, I want to collect images of different waste types so that the model can learn from real-world data.	Images from multiple categories are collected and stored properly in folders.
Data Preparation	As a developer, I want to preprocess and label the images so that they can be used effectively for training.	Images are resized, cleaned, and annotated with correct labels.
Environment Setup	As a developer, I want to set up required tools so that development work can be done smoothly.	All required libraries, frameworks, and tools are installed and working correctly.

3.1.2 Functional Document

Introduction

This sprint covers the early phase of the project, where raw data is gathered and prepared for further use. It also includes setting up the required development environment. This step is important because the quality of data and setup directly influences how well the system performs in later stages.

Product Goal

The aim of this sprint is to create a properly organized dataset and make sure the development setup is ready for model building. A well-arranged dataset helps the model learn better and reduces issues during the training process.

Demography (Users)

- Developers involved in preparing data and configuring the system
- Team members responsible for tracking progress and coordination

Locations

- Local machines used for development work
- Storage locations for datasets (either local folders or cloud storage, if used)

Business Process

In this sprint, the focus is on preparing the dataset so it can be directly used for training. The process begins with collecting images and continues until the data is labeled and stored in an organized manner.

1) Data Collection

- Images are collected from online sources or captured manually
- A wide range of waste types is included to improve diversity

2) Data Sorting

- Images are grouped into categories such as plastic, paper, metal, and others
- Each category is maintained in separate folders for better organization

3) Data Preprocessing

- Images are adjusted to a uniform size and format
- Unwanted noise or irrelevant data is removed to improve clarity

4) Data Annotation

- Objects in the images are marked using bounding boxes
- Each object is labeled based on its waste category

5) Data Storage

- The final dataset is saved in a structured format
- Data is prepared and kept ready for the training stage

Features

1. Image Collection Feature

The system gathers waste images from different sources to ensure variety in the dataset, which helps in improving learning.

2. Data Categorization Feature

The system arranges images into separate waste categories, making it easier for the model to learn and classify correctly.

3. Preprocessing Feature

The system standardizes image size and format, which improves the quality of input data provided to the model.

4. Annotation Feature

The system assigns labels and bounding boxes to images, making them suitable for training the detection model.

Authorization Matrix

Table 3.2 Authorization Matrix

Role	Access Level
Developer	Full access to dataset (create, update, delete)
Admin	Read-only access for monitoring

Assumptions

- The dataset collected is sufficient for initial training
- The system has enough storage and processing capability
- Required tools and libraries are compatible with the system
- Internet access is available for downloading additional data.

3.1.3 Architecture Document

In this sprint, the application works mainly as a data preparation system. It takes raw image inputs, processes them step by step, and converts them into a structured dataset. This dataset is then used in the next sprint for training the month

Table 3.3 Services and Responsibilities

Service	Responsibility
Data Collection Module	Collect images from sources
Preprocessing Module	Clean and resize images
Annotation Module	Label images correctly
Storage Module	Save dataset for training

The data collection module gathers waste images from different sources, which are then cleaned and resized in the preprocessing stage to ensure consistency. After that, the annotation module labels each image correctly so the model can learn from accurate data. Finally, the storage module keeps the prepared dataset organized and ready for training.

System Architecture

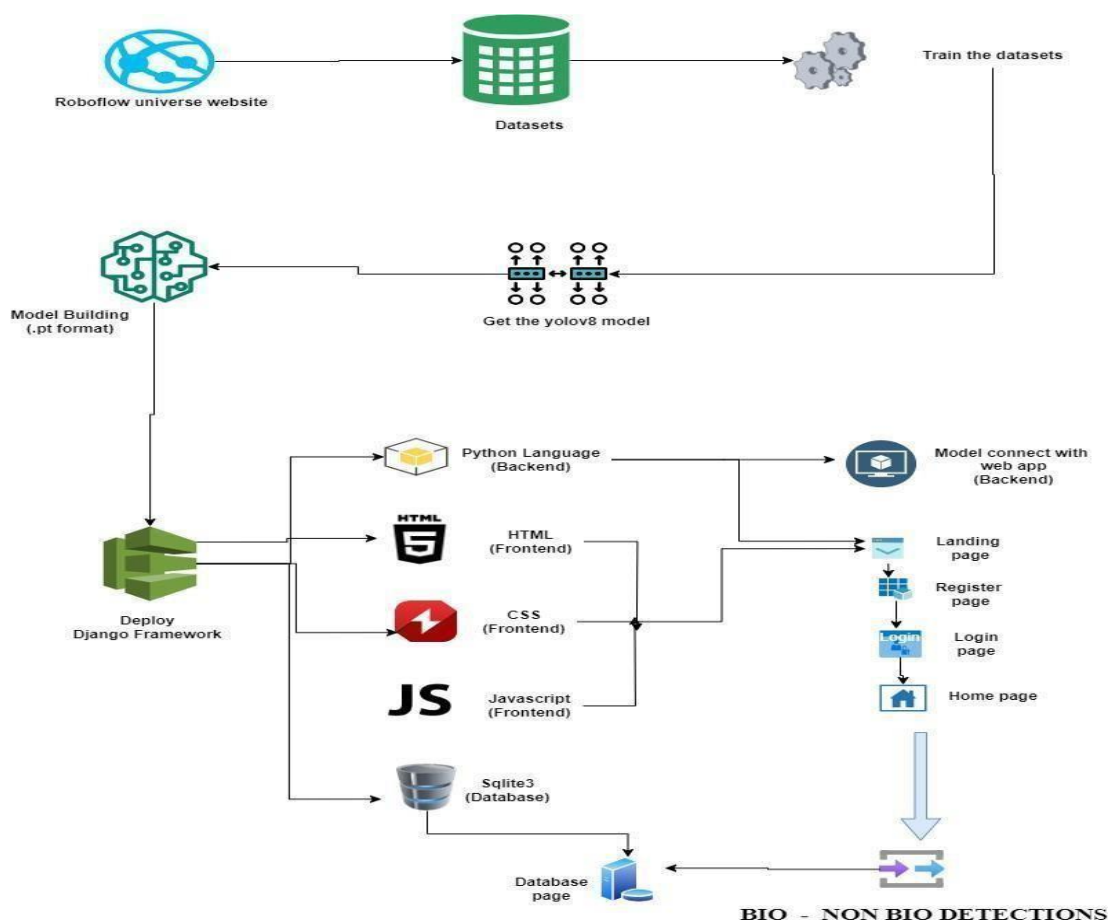


Figure 3.1 System Architecture

The system follows a simple pipeline where waste images are collected from Roboflow and prepared as datasets for training. These datasets are used to train a YOLOv8 model to detect and classify waste. The trained model is then connected to the backend using Python and deployed with the Flask framework. The frontend is created using HTML, CSS, and JavaScript, providing basic pages like login and home. A SQLite database stores user data and results. Users can upload images, and the system processes them to show whether the waste is biodegradable or non-biodegradable.

Data exchange contract

The data exchange in this sprint defines how information flows between different modules involved in dataset preparation. Since this stage mainly deals with image handling, the exchange process is simple and mostly file-based. Proper handling ensures that data moves correctly from one stage to another without loss or errors.

1) Data Flow Handling

- Images collected from sources are transferred to the preprocessing module in batches
- Processed images are passed internally to the annotation module without delay

2) Data Storage and Access

- Annotated data is stored in structured folders for easy access
- The stored dataset is made available to the training module when required

Table 3.4 Data exchange contract

Source	Target	Mode	Frequency
Image Source	Preprocessing Module	File Transfer	Batch
Preprocessing	Annotation Module	Internal	Continuous
Annotation	Storage	File Save	Batch
Storage	Training Module	File Access	On Demand

3.1.4 Outcomes of objective

At the end of Sprint I, the dataset was prepared and arranged in a structured manner. The images were properly categorized and labeled, making them suitable for the training phase. In addition, the development environment was configured successfully, which allowed the next stages of the project to proceed without major issues.

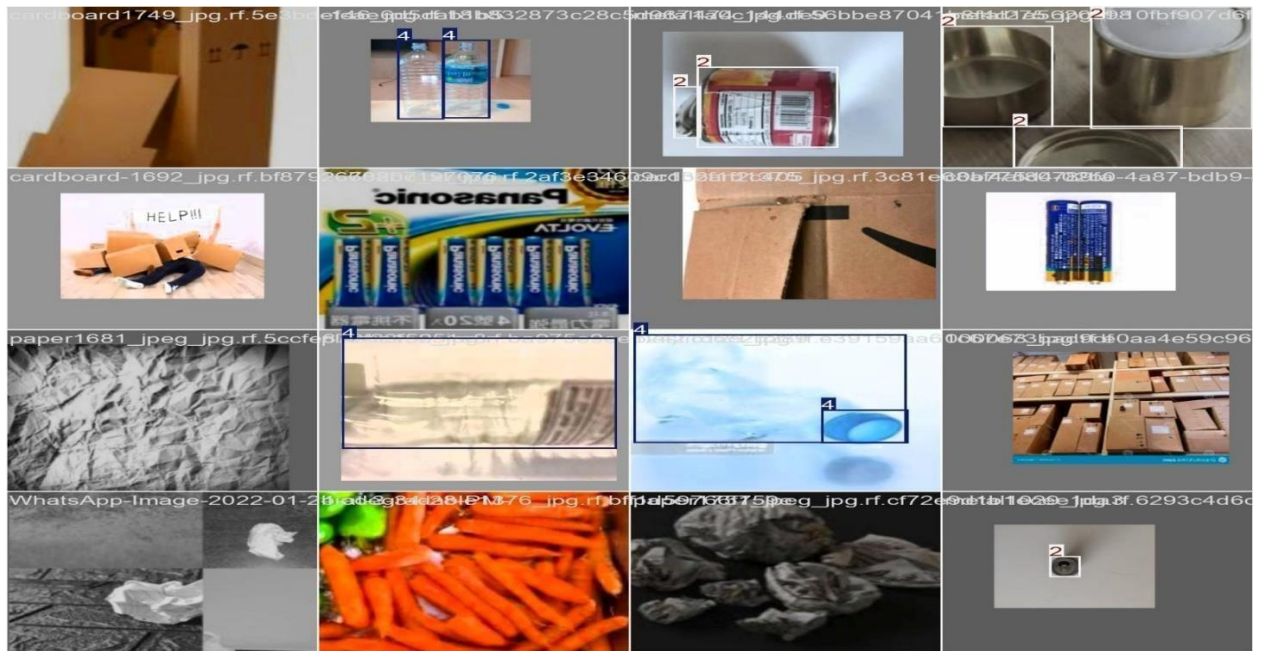


Figure 3.2 Waste Dataset Categories

The figure 3.2 shows a collection of different waste items such as cardboard, plastic, metal containers, batteries, and paper is shown with bounding boxes for detection. The image represents labeled data used for training a waste classification model. It highlights how various materials are identified for sorting and analysis.

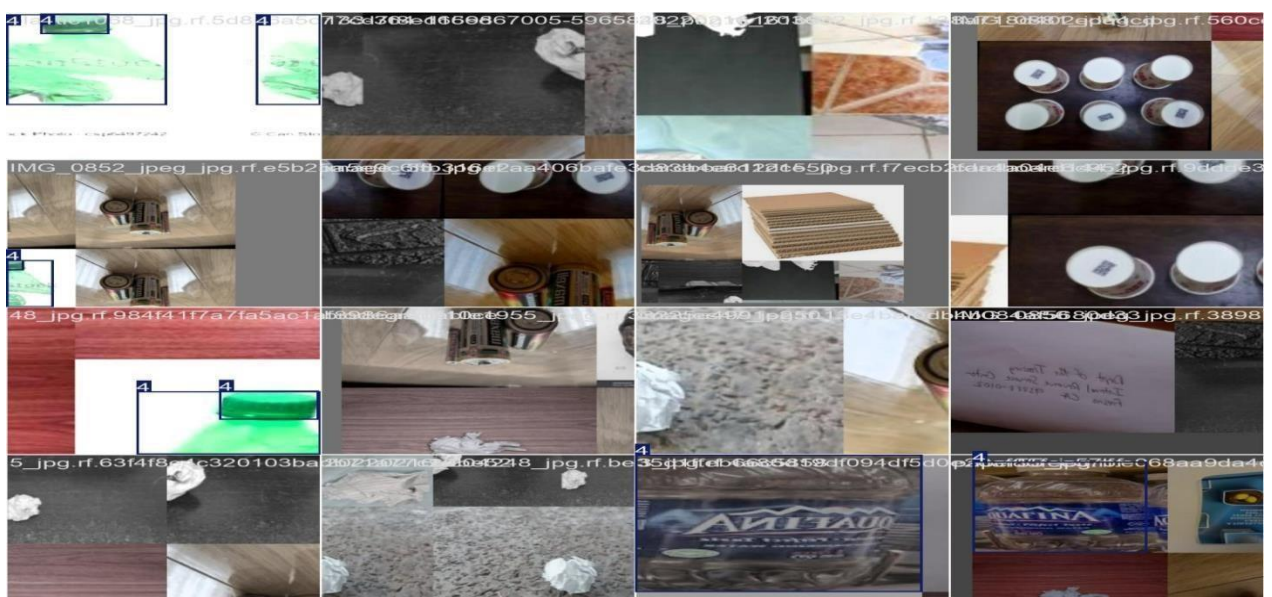


Figure 3.3 Augmented Waste Image Variations

The figure 3.3 shows multiple waste items like bottles, paper pieces, cardboard, and containers marked with detection boxes. It represents sample data used for identifying and classifying different types of waste. The labeled objects help in training the model for accurate recognition.



Figure 3.4 Annotated Waste Images

The figure 3.4 displays various waste materials such as plastic bottles, cardboard, metal cans, and paper items highlighted with bounding boxes. It shows labeled samples used for training and testing the waste detection model. These annotations help the system learn how to identify and classify different waste types accurately.

3.1.5 Sprint Retrospective

The figure 3.5 shows a sprint retrospective table summarizing what went well, challenges faced, ideas for improvement, and action plans. It highlights key observations from the sprint, including data preparation, team coordination and tool usage. The table helps in identifying improvements for better performance in future sprints.

Sprint Retrospective				
What went well	What went poorly	What ideas do you have	How should we take action	
<i>This section highlights the successes and positive outcomes from the sprint. It helps the team recognize achievements and identify practices that should be continued.</i>	<i>This section identifies the challenges, roadblocks, or failures encountered during the sprint. It helps pinpoint areas that need improvement or change.</i>	<i>This section is for brainstorming new approaches, tools, or strategies to enhance the team's efficiency, productivity, or project outcomes.</i>	<i>This section outlines specific steps or solutions to address the issues and implement the ideas discussed, ensuring continuous improvement in future sprints.</i>	Guidelines
Data collection was completed on time with proper categorization.	Some images were unclear or of low quality.	Improve dataset by adding more high-quality and diverse images.	Filter images carefully and remove low-quality data before using it.	Always verify dataset quality before moving to next stage.
Environment setup was smooth and tools were installed successfully.	Initial confusion while using annotation tools.	Learn better tools and techniques for annotation.	Practice tools in advance and refer to proper documentation.	Ensure team members are trained on tools before usage.
Team coordination helped complete tasks efficiently.	Limited number of images in some categories.	Increase dataset size for better balance.	Plan data collection properly and ensure equal distribution.	Maintain proper planning before starting tasks.
Data preprocessing was done correctly.	Some images required manual correction.	Automate preprocessing tasks.	Use scripts and tools for resizing and cleaning images.	Prefer automation to reduce manual effort.
Dataset organization was clear and manageable.	Folder structure was not planned initially.	Maintain structured folder system.	Follow standard naming conventions and hierarchy.	Keep data organized for easy access and reuse.
Work distribution reduced workload.	Labeling process took more time than expected.	Use faster or semi-automatic annotation tools.	Allocate time properly and use efficient tools.	Optimize time management and tool usage.

Figure 3.5 Sprint Retrospective

3.2 Model Development Phase

3.2.1 Objectives with User Stories of Sprint II

This sprint is mainly focused on developing the central component of the system, which is the detection model. The dataset prepared in the previous sprint is utilized to train the model and assess how well it performs under different conditions.

Table 3.5 Objectives of Userstories

Objective	User Story	Acceptance Criteria
Model Development	As a developer, I want to implement a YOLO model so that waste can be detected automatically.	YOLO model is implemented and ready for training.
Model Training	As As a developer, I want to train the model using the prepared dataset	Model is trained successfully with dataset.
Model Testing	As a user, I want accurate predictions so that waste classification is reliable.	Model gives correct predictions on test images.

3.2.2 Functional Document

Introduction

This sprint concentrates on developing the detection model using the dataset prepared in the earlier phase. The focus is on training the model so that it can recognize and categorize different types of waste with improved accuracy.

Product Goal

The objective of this sprint is to create a trained model capable of identifying waste objects in images with reliable performance and efficient processing speed.

Demography (Users)

- Developers working on model training
- Users who will use detection system

Locations

- Local system for training
- GPU/CPU environment for model processing
-

Business Process

In this sprint, the focus is on transforming the prepared dataset into a functional detection model through training, testing, and continuous improvement of its performance.

1) Model Initialization

At this stage, the YOLO algorithm is selected and configured by setting the necessary parameters required for training.

2) Model Training

The dataset is provided as input to the model, allowing it to learn patterns from the labeled images and improve its ability to recognize different types of waste.

3) Model Validation

The model is evaluated using validation data to check how well it performs. Metrics such as accuracy and loss.

4) Model Testing

Test images are used to verify the model's predictions and ensure that it correctly identifies and classifies waste objects.

5) Performance Evaluation

The overall performance of the model is analyzed based on its accuracy and speed, and any necessary improvements are identified.

Features

1) Object Detection Feature

This feature enables the system to identify waste objects present in images and detect multiple items simultaneously.

2) Classification Feature

The system categorizes detected objects into different waste types, which helps improve sorting efficiency.

3) Real-Time Processing Feature

The system is designed to handle image processing with minimal delay, making it suitable for real-time waste detection tasks.

4) Performance Evaluation Feature

This feature helps in measuring key performance indicators such as accuracy and loss, allowing further refinement of the model.

Authorization matrix

Table 3.6 Authorization Matrix

Role	Access Level
Developer	Complete access to model training and testing
Admin	Read-only access to results

The authorization matrix defines the access permissions for different roles in the system. The developer is given complete access, allowing them to handle model training and testing activities. In contrast, the admin has limited access and can only view the results without making any changes, ensuring better control and security.

Assumptions

- Dataset is properly labeled and sufficient for training
- System has enough processing power (CPU/GPU)
- YOLO model is compatible with system
- Training parameters are properly configured

3.2.3 Architecture Document

During this sprint, the application functions as a training system for the detection model. It accepts the labeled dataset as input and processes it using the YOLO algorithm to generate a trained model capable of identifying waste objects.

Table 3.7 Services and Responsibilities

Service	Responsibility
Training Module	Train model using dataset
Validation Module	Check model performance
Testing Module	Test model with new images
Output Module	Generate predictions

System Architecture

The system follows a simple pipeline where waste images are collected from Roboflow and prepared as datasets for training. These datasets are used to train a YOLOv8 model to detect and classify waste. The trained model is then connected to the backend using Python and deployed with the Flask framework. The frontend is created using HTML, CSS, and JavaScript, providing basic pages like login and home. A SQLite database stores user data and results. Users can upload images, and the system processes them to show whether the waste is biodegradable or non-biodegradable

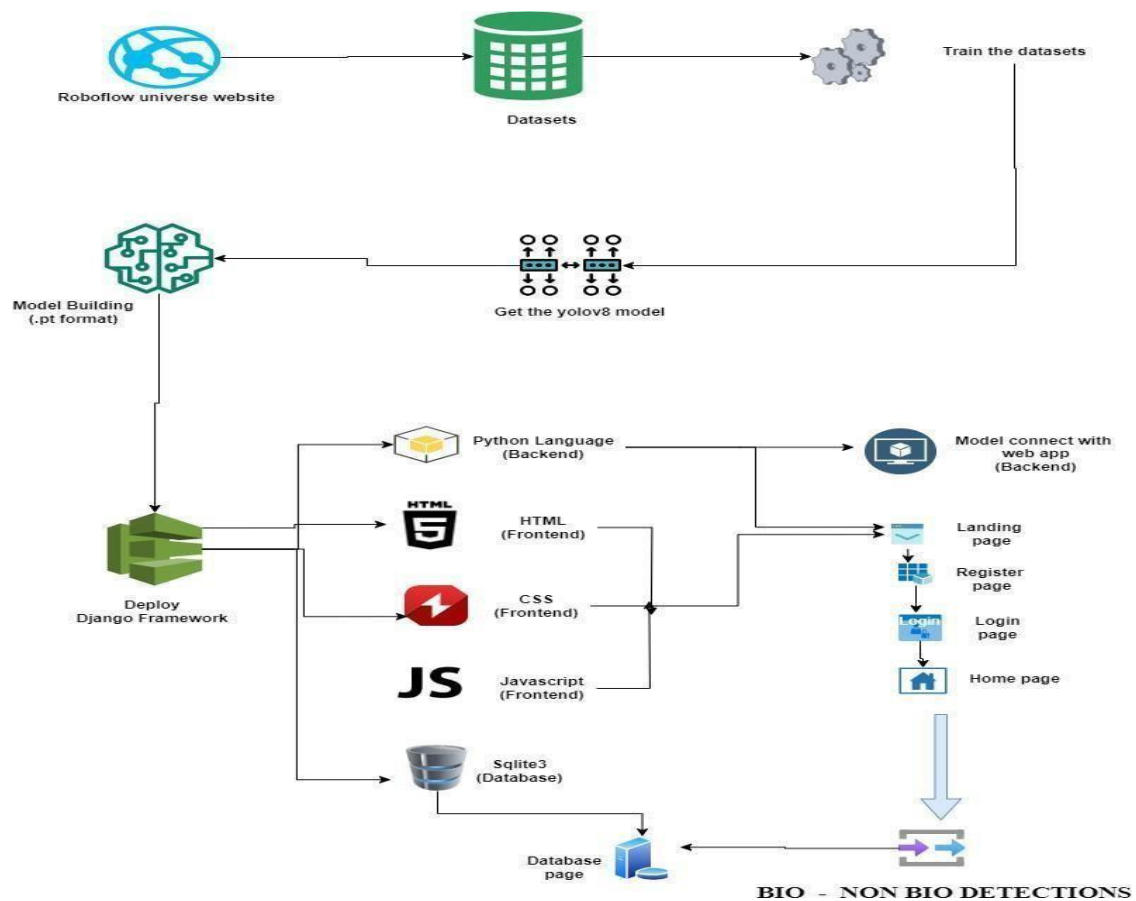


Figure 3.6 System Architecture

Data Exchange Contract

In this stage, data is transferred between different modules involved in training and testing. This ensures that the workflow remains smooth during model development and evaluation.

1) Training Data Flow

- The dataset is provided to the training module in batches
- The model uses labeled images to learn and improve its predictions

2) Output Handling

- The model generates predictions based on the input data
- The results are stored for further evaluation and analysis

Table 3.8 Data Exchange Contract

Source	Target	Mode	Frequency
Dataset	Training Module	Batch Input	Continuous
Training Module	Validation Module	Internal	Periodic
Model	Testing Module	File Access	On Demand
Testing Module	Output Storage	File Save	Batch

3.2.4 Outcome of Objectives

At the completion of Sprint II, the model was trained and evaluated using the prepared dataset. The YOLO-based model demonstrated the ability to detect and categorize waste objects with reliable performance. Testing with sample images showed that the model produced consistent and acceptable results, indicating that the system is functioning as expected.

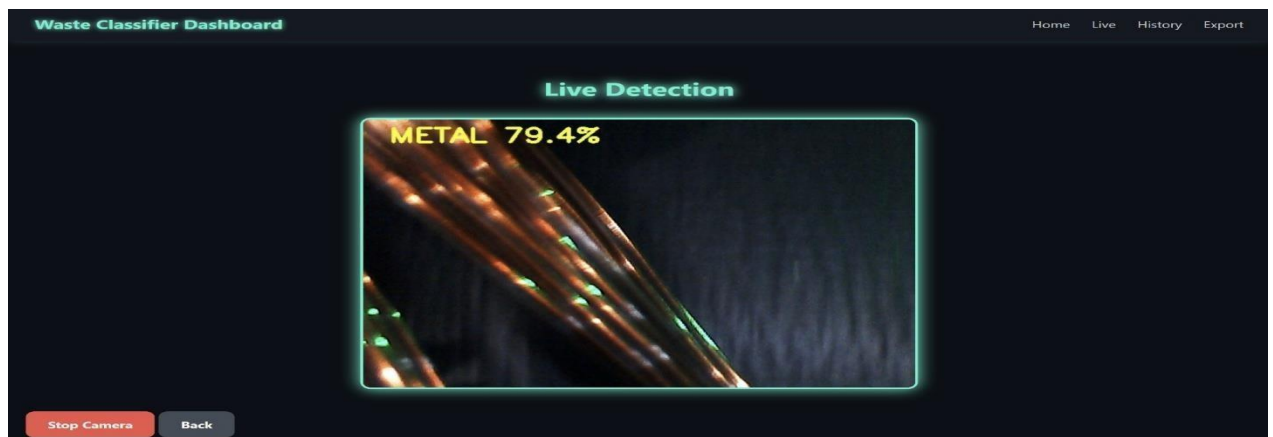


Figure 3.7 High Confidence Detection



Figure 3.8 Low Confidence Detection

3.2.5 Sprint Retrospective

Sprint Retrospective				
What went well	What went poorly	What ideas do you have	How should we take action	
<i>This section highlights the successes and positive outcomes from the sprint. It helps the team recognize achievements and identify practices that should be continued.</i>	<i>This section identifies the challenges, roadblocks, or failures encountered during the sprint. It helps pinpoint areas that need improvement or change.</i>	<i>This section is for brainstorming new approaches, tools, or strategies to enhance the team's efficiency, productivity, or project outcomes.</i>	<i>This section outlines specific steps or solutions to address the issues and implement the ideas discussed, ensuring continuous improvement in future sprints.</i>	<i>Guidelines</i>
Model training was completed successfully.	Training took more time than expected.	Optimize training parameters.	Adjust batch size and epochs for efficiency.	Monitor training time and resource usage.
YOLO model worked well for detection.	Accuracy was lower in some cases.	Improve dataset quality.	Add more training data for better learning.	Focus on data quality for better accuracy.
Testing was done with multiple images.	Some incorrect predictions were observed.	Fine-tune the model.	Adjust hyperparameters and retrain.	Perform regular testing and improvements.
Model handled multiple objects.	Difficulty in detecting small objects.	Improve detection for small objects.	Use higher resolution images.	Ensure dataset includes small object samples.
System performance was stable.	Resource usage was high during training.	Optimize system performance.	Use GPU if available.	Manage system resources efficiently.
Workflow was properly followed.	Initial setup of training parameters was confusing.	Document training steps clearly.	Maintain proper documentation.	Follow structured approach in model training.

Figure 3.9 Sprint Retrospective

The figure 3.9 shows a sprint retrospective summarizing what worked well, issues faced, and possible improvements. It highlights successful model training and stable performance, while noting challenges like lower accuracy and high resource usage. It also suggests actions such as tuning parameters, improving data quality, and maintaining proper documentation for better results

3.3 Application Integration Phase

3.3.1 Objectives with User Stories of Sprint III

This sprint is focused on bringing together all the components developed in earlier stages to form a complete and functional system. The trained model is integrated with a web application so that users can easily interact with the system and view the output results.

Table 3.9 Objectives of User Stories

Objective	User Story	Acceptance Criteria
System Integration	As a developer, I want to connect the trained model with the backend so that all parts of the system work together smoothly.	Model is successfully connected with the application backend.
Web Application Development	As a user, I want a web interface so that I can interact with the system.	Web interface is developed and accessible.
Result Display	As a user, I want to see detection results clearly so that I can understand output.	Results are displayed properly with labels and bounding boxes.

3.3.2 Functional Document

Introduction

This sprint represents the final phase of development, where all individual components are brought together to form a complete system. The trained model is integrated with a web application so that users can interact with it and easily view the results.

Product Goal

The aim of this sprint is to build an easy-to-use system that accepts image inputs and displays waste detection results in a clear and understandable format for users.

Demography (Users)

- End users who use the system for waste detection
- Developers managing system functionality

Locations

- Local system hosting the web application
- Browser interface for user interaction

Business Process

In this stage, the focus is on linking the trained model with the user interface and enabling smooth interaction between the system and the user in real time.

1) User Input Handling

The user provides an image as input, which is accepted by the system and prepared for further processing.

2) Backend Processing

The input image is sent to the backend server, where it is processed and passed to the trained model.

3) Model Prediction

The YOLO model analyzes the image and detects objects, producing outputs with labels and bounding boxes.

4) Result Processing

The generated output is organized and relevant details are extracted for display.

5) Output Display

The final results are shown through the web interface, allowing users to clearly understand the detected waste categories.

Features

1) User Interface Feature

This feature provides a simple interface that allows users to upload images and interact with the system easily.

2) Model Integration Feature

The trained model is connected to the backend, ensuring smooth communication and proper data flow.

3) Result Visualization Feature

The system presents detection results using labeled outputs and visual indicators, making it easier to interpret the identified waste types.

4) System Interaction Feature

This feature enables users to interact with the system efficiently and receive quick responses along with the output.

Authorization Matrix

Table 3.10 Authorization Matrix

Role	Access Level
Developer	Full access to system integration and backend
User	Limited access (can upload and view results)

The authorization matrix outlines the access control for different roles in the system. The developer has full access, allowing them to manage system integration and handle backend operations. The user is given limited access, where they can upload data and view the results, but do not have permission to modify or control the system functions.

Assumptions

- Trained model is working correctly before integration
- Web framework is properly configured
- System can handle input and output without delay
- Users have basic knowledge to interact with interface
-

3.3.3 Architecture Document

In this sprint, the application functions as a complete system where the frontend, backend, and trained model are integrated. Users interact with the system through a web interface, while the backend handles the requests and processes them using the trained model to generate the final output.

Services and Responsibilities

Table 3.11 Services and Responsibilities

Service	Responsibility
Frontend Interface	Accept user input and display results
Backend Server	Process requests and connect modules
Model Module	Perform detection and classification
Output Module	Format and display results

System Architecture

The system follows a simple pipeline where waste images are collected from Roboflow and prepared as datasets for training. These datasets are used to train a YOLOv8 model to detect and classify waste. The trained model is then connected to the backend using Python and deployed with the Flask framework. The frontend is created using HTML, CSS, and JavaScript, providing basic pages like login and home. A SQLite database stores user data and results. Users can upload images, and the system processes them to show whether the waste is biodegradable or non-biodegradable

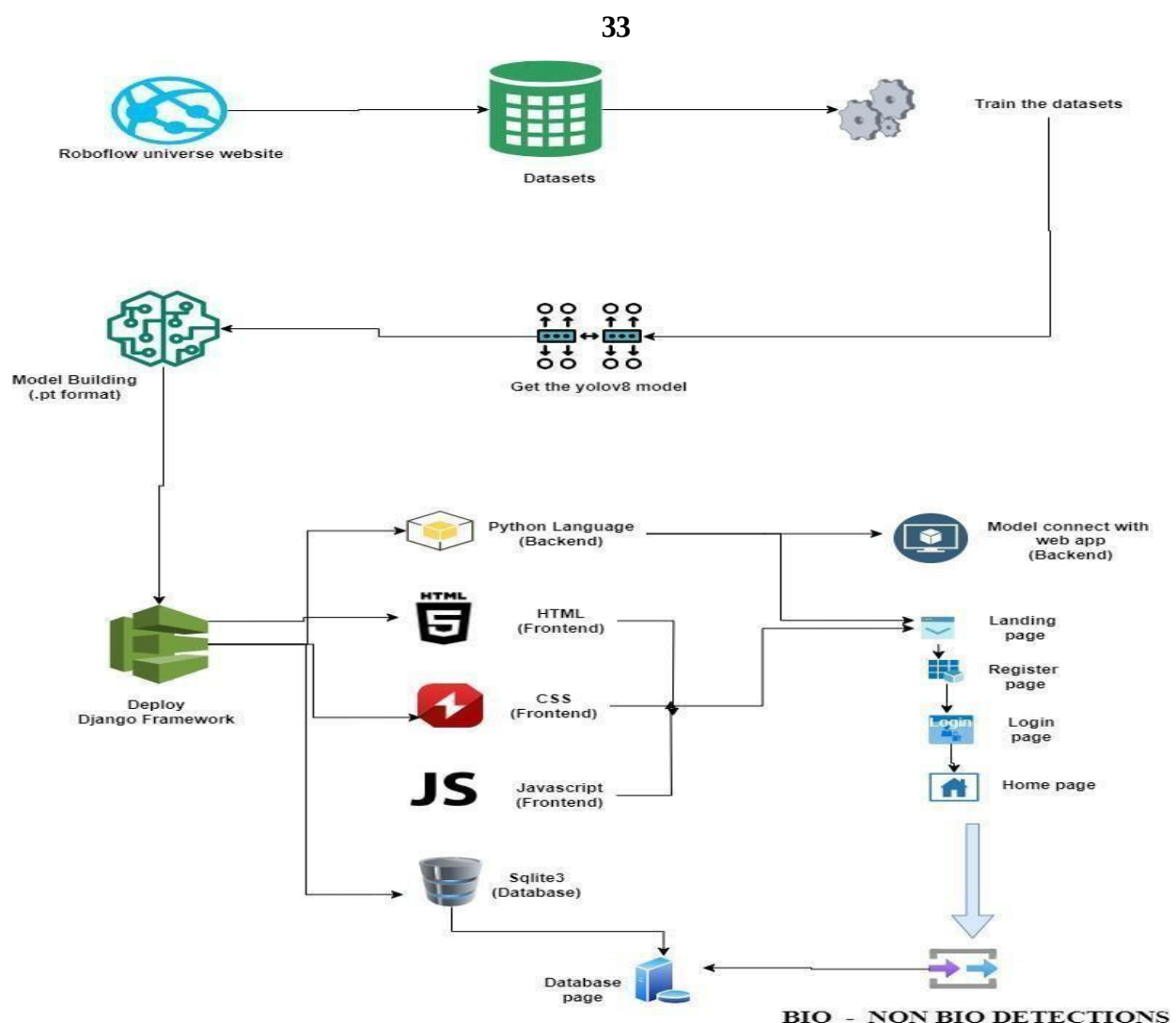


Figure 3.10 System Architecture

Data Exchange Contract

In this sprint, data is exchanged between the user interface, backend system, and the trained model. This ensures that user inputs are processed correctly and the results are returned smoothly without delays.

1) Input Data Flow

- The user provides an image through the web interface
- The input image is forwarded to the backend server for further processing

2) Output Data Flow

- The model analyzes the image and generates the detection output
- The processed result is sent back to the frontend for display

Data Exchange Table

Table 3.12 Data Exchange Table

Source	Target	Mode	Frequency
User Interface	Backend Server	HTTP Request	On Demand
Backend Server	Model Module	Internal	Continuous
Model Module	Backend Server	Internal	Continuous
Backend Server	User Interface	HTTP Response	On Demand

The data exchange table describes how information flows between different parts of the system. The user interface communicates with the backend server through HTTP requests whenever needed, and the server responds in the same way. Inside the system, the backend server and model module continuously exchange data to process inputs and generate results. This setup ensures smooth and timely communication between all components.

3.3.4 Outcome of Objectives

By the end of this sprint, all components were successfully integrated into a working system. The trained model was connected with the web application, allowing users to upload images and view detection outputs easily. The system operated reliably under normal conditions and presented results in a clear and user-friendly manner. This marked the completion of the overall system development.

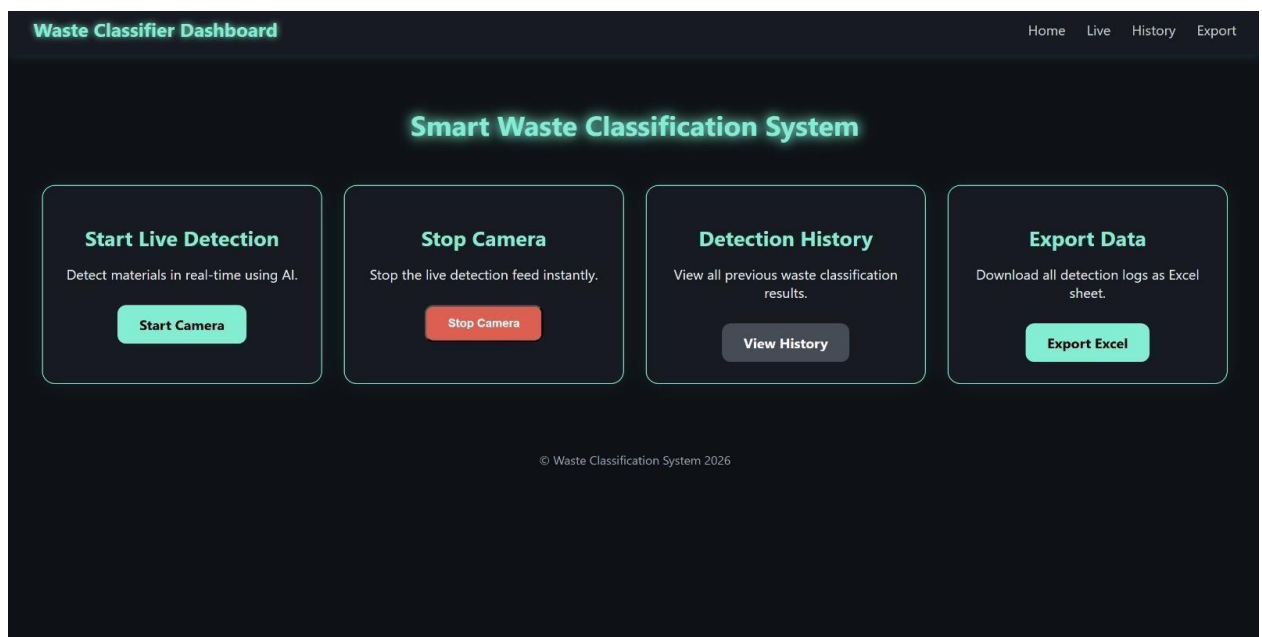


Figure 3.11 Main Dashboard Screen

The figure 3.11 shows the dashboard of a Smart Waste Classification System designed for easy user interaction. It includes options to start live detection using a camera, stop the camera feed, view past detection results, and export data as an Excel file. The interface is simple and organized, allowing users to quickly access key features and manage waste classification tasks efficiently.



Figure 3.12 Low Accuracy Live Output

The figure 3.12 shows the live detection screen of the Smart Waste Classification System. It displays a real-time camera feed where the system is trying to identify an object, but the result is marked as “Low Confidence (47.8%),” indicating that the model is not fully certain about the prediction. The interface also includes options to stop the camera and go back, allowing the user to control the detection process easily.

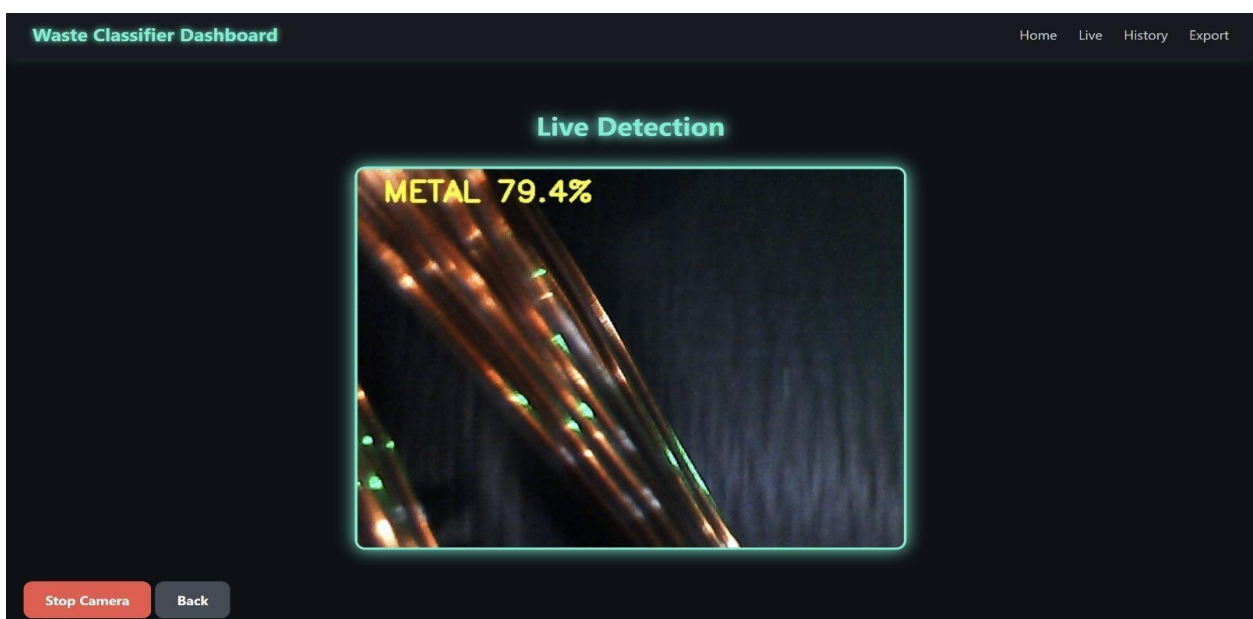


Figure 3.13 Live Waste Detection

The figure 3.13 shows the live detection feature of the Smart Waste Classification System in action. The system is capturing a real-time image and has identified the object as metal with a confidence level of 79.4%, indicating a fairly reliable prediction. The interface clearly displays the result on the screen along with options to stop the camera or go back, making it easy for the user to control the process.



The screenshot shows a web application titled "Waste Classifier Dashboard". In the top right corner, there are navigation links: "Home", "Live", "History", and "Export". The main content area is titled "Detection History" in a large, bold, green font. Below this title is a table with four columns: "id", "material", "confidence", and "timestamp". The table contains 11 rows of data, all of which are classified as "METAL". The confidence levels range from approximately 85.4% to 90.0%, and the timestamps are from 2026-03-03 08:20:37 to 2026-03-03 08:20:43.

id	material	confidence	timestamp
1	METAL	87.352395	2026-03-03 08:20:37
2	METAL	87.754583	2026-03-03 08:20:37
3	METAL	88.196248	2026-03-03 08:20:38
4	METAL	86.664081	2026-03-03 08:20:39
5	METAL	90.005016	2026-03-03 08:20:39
6	METAL	88.858318	2026-03-03 08:20:40
7	METAL	88.467503	2026-03-03 08:20:41
8	METAL	86.852741	2026-03-03 08:20:41
9	METAL	86.027652	2026-03-03 08:20:42
10	METAL	87.463558	2026-03-03 08:20:42
11	METAL	85.433626	2026-03-03 08:20:43

Figure 3.14 Saved Detection History

The figure 3.14 shows the detection history section of the Smart Waste Classification System. It displays a table containing past results, including details such as the ID, detected material, confidence level, and timestamp for each entry. This allows users to track previous classifications and review the system’s performance over time in a clear and organized manner.

3.3.5 Sprint Retrospective

Sprint Retrospective				
What went well	What went poorly	What ideas do you have	How should we take action	
<i>This section highlights the successes and positive outcomes from the sprint. It helps the team recognize achievements and identify practices that should be continued.</i>	<i>This section identifies the challenges, roadblocks, or failures encountered during the sprint. It helps pinpoint areas that need improvement or change.</i>	<i>This section is for brainstorming new approaches, tools, or strategies to enhance the team's efficiency, productivity, or project outcomes.</i>	<i>This section outlines specific steps or solutions to address the issues and implement the ideas discussed, ensuring continuous improvement in future sprints.</i>	<i>Guidelines</i>
System integration was completed successfully.	Minor bugs occurred during integration.	Improve testing before integration.	Perform testing at each stage.	Follow step-by-step integration process.
Web interface was developed and working properly.	UI design was basic.	Improve user interface design.	Enhance UI using better design tools.	Focus on user-friendly design.
Model worked well with backend.	Delay in response time in some cases.	Optimize system performance.	Reduce processing time and improve efficiency.	Monitor system performance regularly.
Results were displayed clearly.	Formatting issues in some outputs.	Improve result formatting.	Adjust display settings and layout.	Maintain consistency in output display.
System handled user input correctly.	Error handling was limited.	Add proper error handling.	Implement validation and error messages.	Ensure system handles all cases properly.
Overall system was functional.	Integration took more time than expected.	Plan integration tasks better.	Allocate proper time for integration.	Follow proper planning and scheduling.

Figure 3.15 Sprint Retrospective

This sprint retrospective summarizes the system integration phase, highlighting what worked well and the issues faced. It points out areas like UI improvement, performance optimization, and better error handling. The table also suggests clear actions such as improved testing, planning, and design to enhance future results.

CHAPTER 4

RESULTS AND DISCUSSION

4.1 Project Outcomes

The Smart Waste Classification System developed in this project was assessed based on how effectively it can detect and categorize different types of waste materials. The system combines a trained detection model with a web-based interface, enabling users to interact with it and view results in real time. The outcomes indicate that the system is capable of identifying waste objects under different conditions and providing useful output to the user. The evaluation focuses on aspects such as system performance, behaviour under varying conditions, and overall reliability. These observations help in understanding both the strengths and areas that can be further improved.

Performance Evaluation

The performance of the model was examined using training results and prediction confidence levels. During the training process, accuracy gradually increased while the loss values decreased, showing that the model was learning from the dataset in a stable manner. The training graphs also indicate that the model reached a consistent state after multiple iterations. In practical scenarios, the system produced higher confidence values when the input image was clear and properly aligned. For instance, certain waste types like metal were identified more accurately under stable lighting conditions. This suggests that the model performs better when the input data is clean and well-structured. However, a slight drop in performance was noticed when the input images contained noise, blur, or poor lighting. This highlights that the model's effectiveness is still influenced by the quality of input data. Further improvements can be achieved by training the model with more diverse and complex datasets.

Table 4.1 Performance Evaluation

Metric	Observation	Description
Accuracy	High in clear conditions	The model gives correct predictions when objects are clearly visible and properly positioned.
Confidence Score	Varies with input quality	High confidence for clear images and lower confidence for blurred or unclear inputs.
Training Performance	Stable improvement	Accuracy increased gradually during training while loss decreased, showing effective learning.
Detection Speed	Fast response	The system processes input quickly and provides near real-time results.
Category Performance	Uneven across classes	Some categories like metal performed better due to more training data.

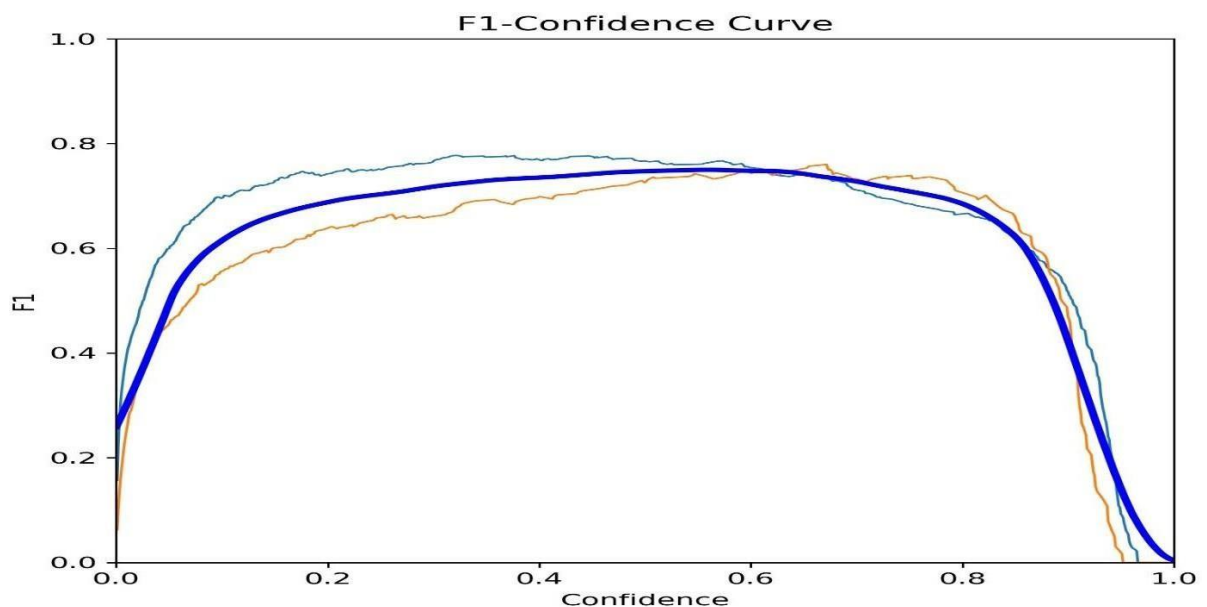


Figure 4.1 Optimal Confidence Curve

The figure 4.1 shows how the F1 score changes with different confidence thresholds. The score increases steadily and reaches a peak at mid confidence levels, then drops at higher thresholds. It indicates the model performs best within an optimal confidence range.

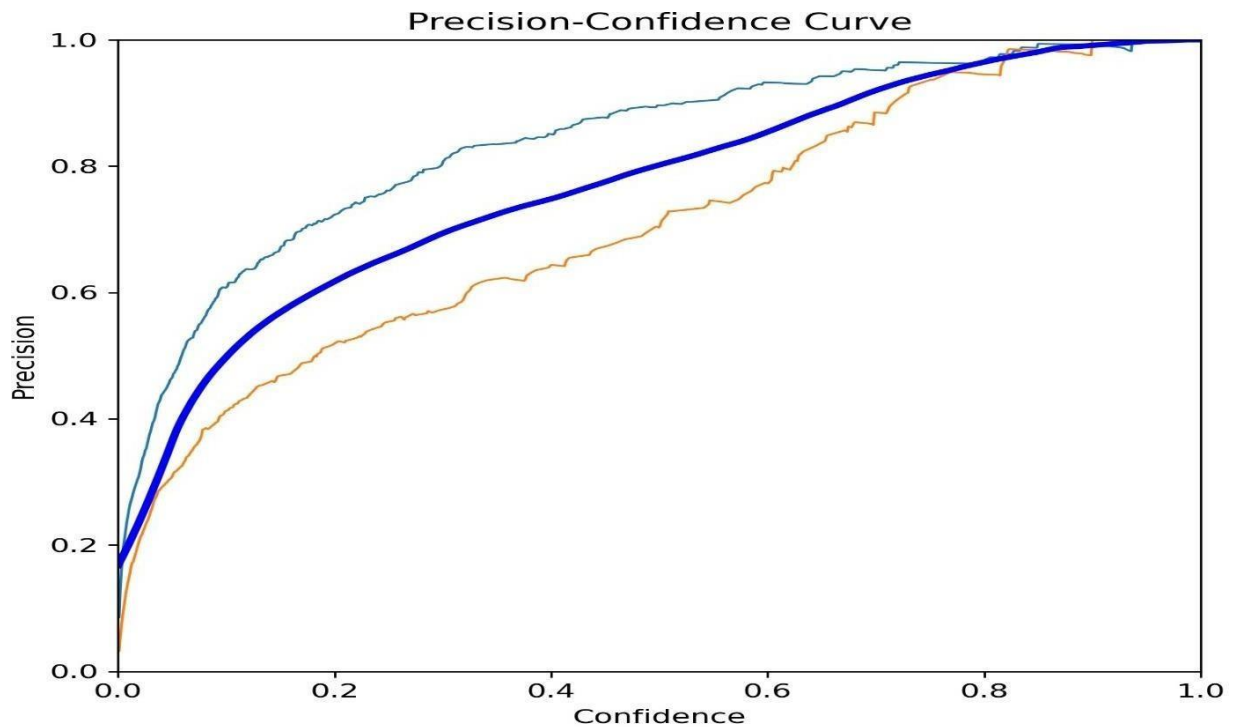


Figure 4.2 Precision vs Confidence

The figure 4.2 shows that precision improves as confidence increases. Higher confidence thresholds lead to more accurate predictions, though fewer detections are made. It highlights the trade-off between precision and coverage in the model. Different test cases were analyzed to understand how the system performs under various conditions. It was observed that the system gives better results when the input images are clear and of higher quality. Images with good resolution produced higher confidence values, whereas blurred or partially visible objects led to lower prediction accuracy. A comparison was also made across different waste categories. Certain types, such as metal and plastic, were identified more accurately, mainly because they were well represented in the training dataset. On the other hand, categories with fewer samples showed comparatively lower performance. These observations indicate that both the quality and diversity of the dataset have a strong

impact on the overall performance of the system. Improving dataset balance can further enhance the accuracy and reliability of predictions.

Table 4.2 Comparison of images taken at different working condition

Scenario	Observation	Result
High-quality image	Object is clear and well-lit	Model gives high confidence and accurate prediction
Low-quality image	Image is blurred or noisy	Confidence decreases and prediction may be incorrect
Proper lighting	Object is clearly visible	Detection performance is strong
Poor lighting	Object is dark or unclear	Accuracy reduces due to lack of visibility
Simple background	Object is easy to identify	Model detects object quickly and correctly
Complex background	Background has multiple objects	Model may get confused and accuracy decreases
Balanced dataset	All categories have enough data	Better overall performance

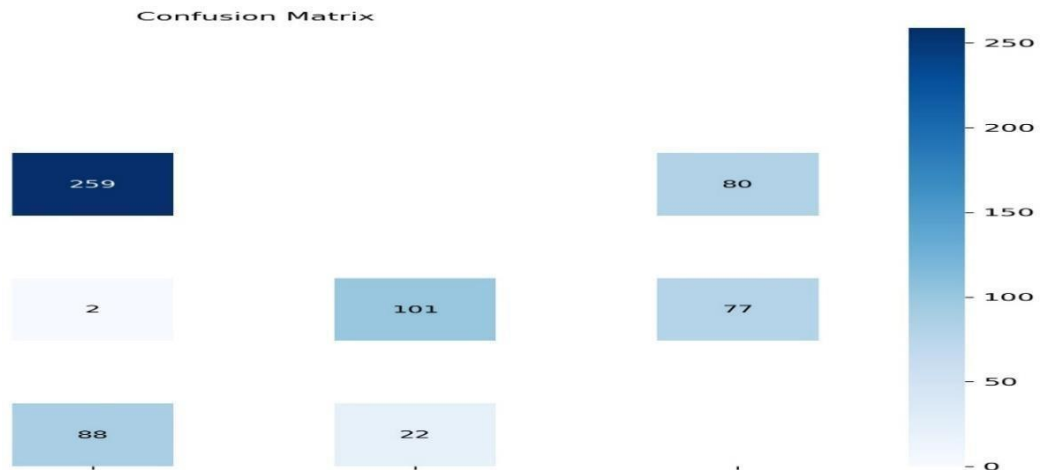


Figure 4.3 Confusion Matrix Counts

This matrix shows how the model classified each category, with correct predictions on the diagonal and errors in other cells. It indicates good performance in some classes, but also shows misclassification between certain categories. This helps identify where the model is getting confused

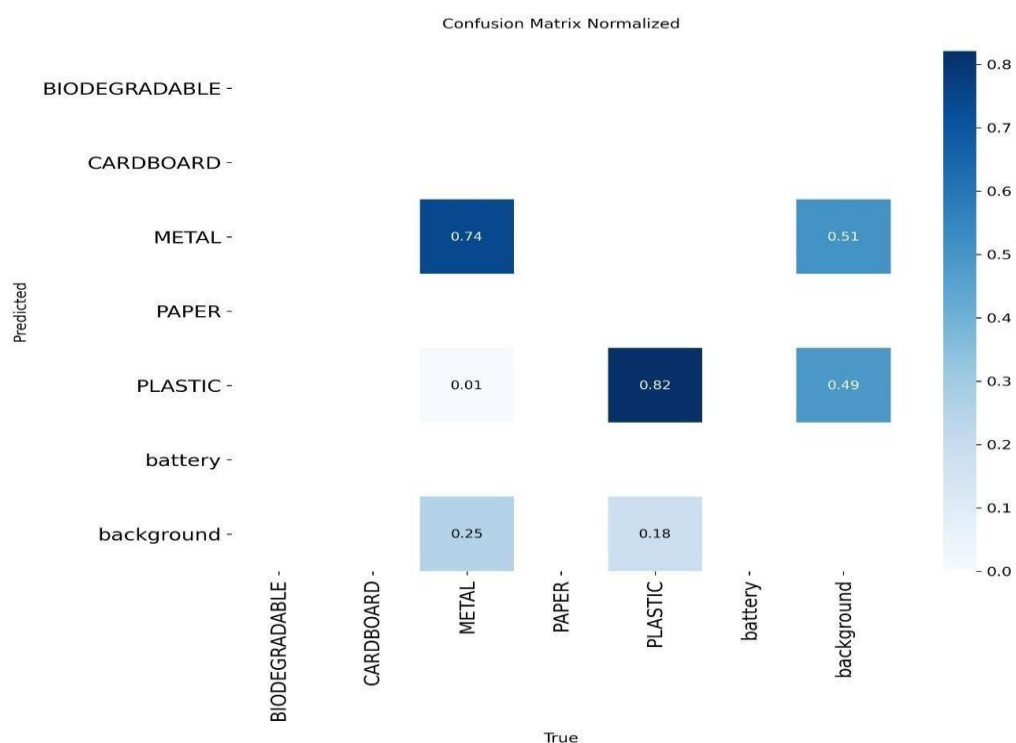


Figure 4.4 Normalized Confusion Matrix

This version presents the results in percentage form, making it easier to compare class-wise performance. It shows that some classes like plastic and metal have higher accuracy, while others have more confusion. This highlights the need for better training data and tuning for weaker classes.

Testing Results

The system was evaluated in real-time using the developed web application. Users were able to initiate detection, upload images, and observe the results without noticeable delay. The responses were generated quickly, and predictions were displayed along with confidence values, showing that the integration between the model and the application is functioning properly. The system also included a feature to store detection history, which allows users to review previous results when needed. This helps in monitoring system performance over time. In addition, the option to export results makes it easier to use the data for further analysis. During testing, the system performed consistently and met the expected requirements. Although minor variations were noticed under challenging conditions, such as poor image quality, the system worked effectively in most practical scenarios.

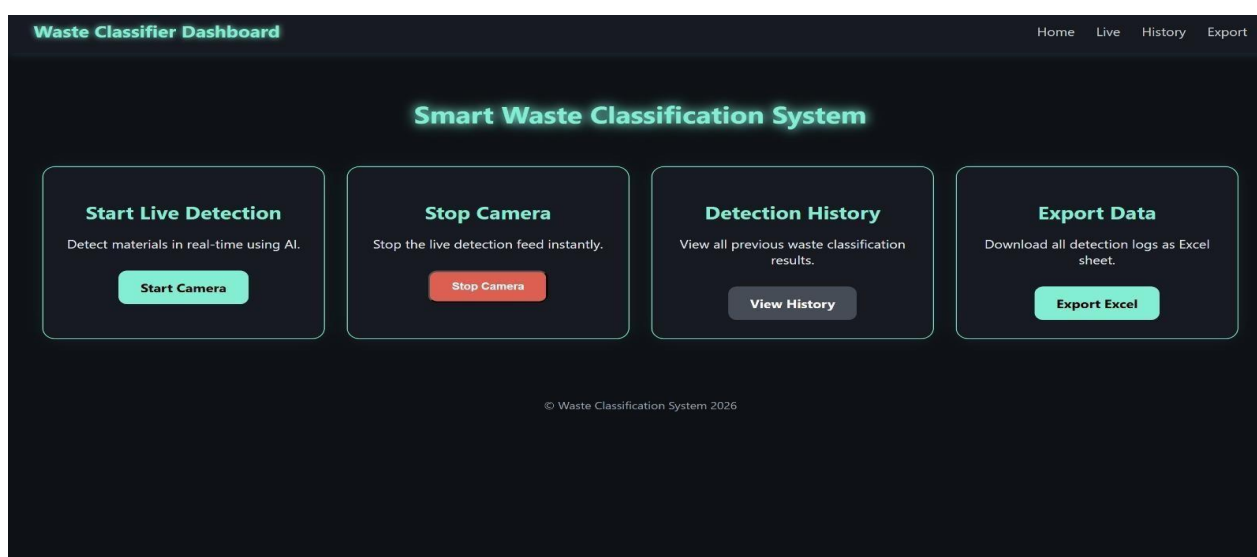


Figure 4.5 System Dashboard Interface

The figure 4.5 provides options to start/stop live detection, view past results, and export data. It is simple and user-friendly, allowing easy control of the waste classification process in real time.

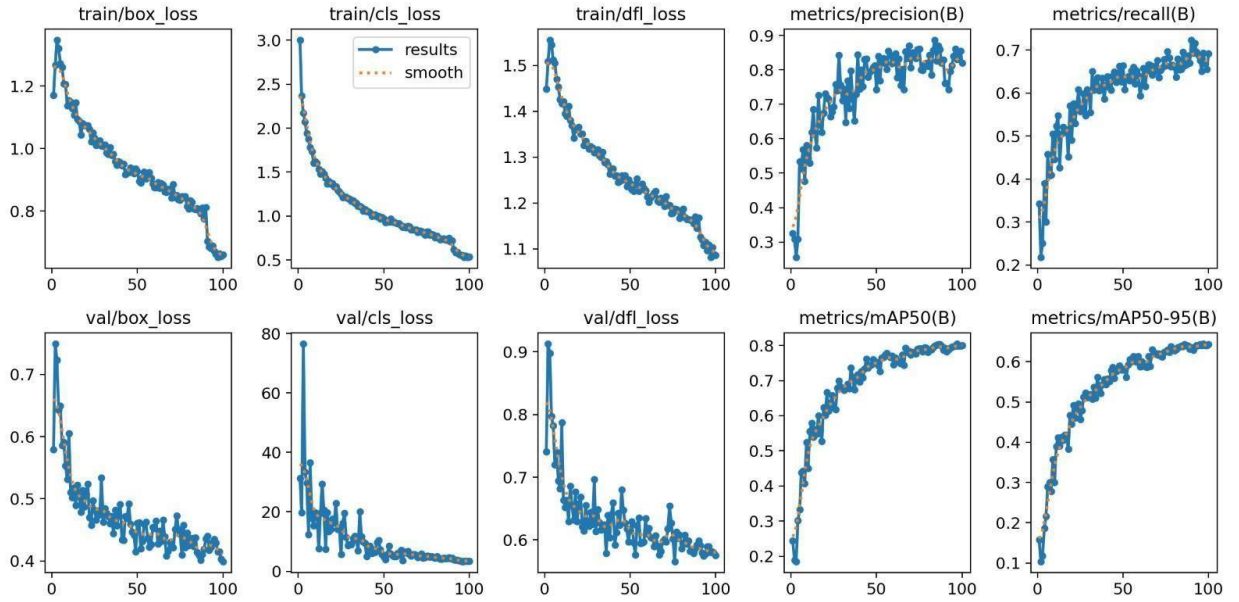


Figure 4.6 Real-Time Detection Output

The figure 4.6 presents the real-time detection output along with the training performance of the model. From the graph, it can be observed that both training and validation losses gradually decrease as the number of epochs increases. This steady reduction indicates that the model is learning effectively and minimizing errors over time. At the same time, key performance metrics such as precision, recall, and mean Average Precision (mAP) show a consistent upward trend. This improvement suggests that the model is becoming more accurate in identifying and classifying waste materials correctly. The increase in precision means that the number of correct positive predictions is improving, while the rise in recall indicates that the model is successfully detecting more relevant objects without missing them. Similarly, the growth in mAP reflects better overall detection performance across different classes. These trends together show that the model is not only learning but also generalizing well to new data.

epoch	time	train/box_loss	train/cls_loss	train/df_l_loss	metrics/precision(B)	metrics/recall(B)	metrics/mAP50(B)	metrics/mAP50-95(B)	val/box_loss	val/cls_loss	val/df_l_loss	lr/pg0	lr/pg1	lr/pg2
1	46.9952	1.17069	3.0052	1.44914	0.32584	0.34339	0.24463	0.15838	0.57921	31.2207	0.74112	0.000332389	0.000332389	0.000332389
2	85.7108	1.26734	2.36703	1.50834	0.30798	0.21785	0.18921	0.10397	0.74908	19.6921	0.91282	0.000659132	0.000659132	0.000659132
3	123.826	1.34812	2.17396	1.556	0.25523	0.25053	0.18519	0.11793	0.72335	76.5317	0.89775	0.000979274	0.000979274	0.000979274
4	161.467	1.32133	2.07275	1.54582	0.3078	0.38983	0.30144	0.18599	0.64289	33.3905	0.79735	0.0009703	0.0009703	0.0009703
5	199.436	1.27302	1.94206	1.51217	0.53468	0.3006	0.33231	0.21649	0.64928	29.656	0.78224	0.0009604	0.0009604	0.0009604
6	237.43	1.2607	1.87518	1.50568	0.51268	0.45807	0.43804	0.2893	0.58544	12.3962	0.71977	0.0009505	0.0009505	0.0009505
7	275.27	1.20769	1.78023	1.46971	0.56931	0.40817	0.44326	0.30003	0.59087	36.5835	0.74001	0.0009406	0.0009406	0.0009406
8	313.01	1.20579	1.73495	1.45367	0.47579	0.40905	0.40666	0.27858	0.55317	17.7198	0.6946	0.0009307	0.0009307	0.0009307
9	350.587	1.13738	1.60098	1.42649	0.58155	0.50502	0.52449	0.35739	0.53125	15.3895	0.68174	0.0009208	0.0009208	0.0009208
10	388.465	1.15329	1.6226	1.41708	0.54254	0.44584	0.44996	0.29977	0.60489	19.7848	0.7876	0.0009109	0.0009109	0.0009109
11	426.003	1.14811	1.6015	1.42094	0.52981	0.52399	0.55507	0.39004	0.51049	18.4604	0.66375	0.000901	0.000901	0.000901
12	463.469	1.12892	1.53208	1.39544	0.61873	0.54783	0.57865	0.41158	0.50207	7.63222	0.65225	0.0008911	0.0008911	0.0008911
13	500.939	1.10791	1.47826	1.38929	0.68542	0.4268	0.53942	0.39126	0.51781	19.6216	0.66328	0.0008812	0.0008812	0.0008812
14	538.66	1.14742	1.50886	1.41127	0.61203	0.505	0.5608	0.41274	0.48959	29.3143	0.62965	0.0008713	0.0008713	0.0008713
15	576.293	1.09422	1.47671	1.38	0.57275	0.52083	0.54811	0.40472	0.52181	20.5246	0.68631	0.0008614	0.0008614	0.0008614
16	613.735	1.08745	1.4307	1.36941	0.72613	0.5097	0.5995	0.41746	0.50398	7.50357	0.65497	0.0008515	0.0008515	0.0008515
17	651.166	1.04347	1.36487	1.34159	0.63954	0.51521	0.55768	0.4177	0.47828	19.7544	0.63235	0.0008416	0.0008416	0.0008416
18	688.484	1.08028	1.3936	1.36022	0.61937	0.45184	0.52793	0.3829	0.51406	13.3904	0.67665	0.0008317	0.0008317	0.0008317
19	725.696	1.07364	1.36409	1.36191	0.67467	0.57077	0.62396	0.46771	0.4865	14.1944	0.62905	0.0008218	0.0008218	0.0008218

Figure 4.7 Stored Detection Results

The figure 4.7 shows the stored detection results in a structured format, providing detailed information recorded during the training process. It includes values for each epoch such as training loss, validation loss, and important evaluation metrics like precision, recall, and mAP. By presenting these values clearly, the figure makes it easier to understand how the model is performing at different stages of training. As the epochs progress, the losses gradually decrease, which indicates that the model is reducing its errors and learning patterns more effectively. At the same time, the evaluation metrics show a steady improvement, meaning the model is becoming more accurate in identifying and classifying objects. This continuous change in values helps in tracking the learning behavior of the model over time. Overall, the figure plays an important role in monitoring the model's development. It confirms that the training process is stable and that the model is improving consistently with each epoch, leading to better performance and more reliable detection results.

CHAPTER 5

CONCLUSION AND FUTURE ENHANCEMENT

5.1 Conclusion

The smart waste classification system developed in this project demonstrates how deep learning techniques can be applied to address real-world challenges in waste management. The model was trained over multiple epochs, and during this process, both training and validation loss showed a gradual decrease, indicating effective learning from the dataset. At the same time, evaluation metrics such as precision, recall, and mean average precision (mAP) improved as training progressed, showing that the model became better at distinguishing between different waste categories.

From the analysis of training graphs and confusion matrix results, it can be observed that the system performs well in most cases and correctly classifies a majority of the waste items. However, some misclassifications were noticed, particularly when objects had similar visual characteristics, which is a common limitation in image-based detection systems. An important achievement of this project is the integration of the trained model with a web-based application, making the system more practical and user-friendly for real-time usage. The interface allows users to upload images, perform live detection, and view results instantly. It also provides confidence scores for predictions, helping users understand the reliability of the output. Additional features such as maintaining detection history and exporting results enhance the overall usability of the system. These functionalities make the system useful not only for demonstration but also for real-world applications such as recycling facilities and smart waste monitoring environments. In conclusion, the project successfully meets its objective of developing an efficient real-time waste classification system. With further improvements and expansion, the system can be adapted for larger-scale deployment and contribute to more effective and automated waste management solutions.

5.2 Future Enhancement

The system can be improved in several ways to enhance its overall effectiveness, reliability, and ease of use. The accuracy of the model can be increased by refining the training process and applying better tuning techniques. Expanding the dataset with a wider variety of images, including different backgrounds, lighting conditions, and object orientations, will help the model perform more consistently in real-world scenarios. The system can also be extended to classify additional categories of waste, making it more adaptable and useful across different environments. Improving processing speed is equally important to ensure smooth and faster real-time detection, especially when the system is used continuously.

Another key area of improvement is strengthening detection performance in challenging conditions such as low-light environments, shadows, or cluttered surroundings. At the same time, efforts should be made to reduce incorrect predictions, particularly for objects that have similar visual features, by improving feature extraction and classification techniques. The user interface can be redesigned to be more intuitive and easy to navigate, allowing users to quickly understand system outputs and interact with the dashboard without confusion. Providing mobile accessibility will further increase convenience, enabling users to monitor and control the system from anywhere.

In addition, features like cloud storage can be integrated to store detection results and allow seamless access to data from multiple locations. This will also support better data management and future analysis. The system can be connected with hardware components such as smart bins or embedded devices to enable real-time implementation in practical settings. It can also be extended to support automated physical sorting of waste materials, reducing manual effort and improving efficiency. Implementing continuous learning will allow the model to adapt and improve over time by learning from new data inputs.

Further improvements include adding multi-language support to make the system accessible to users from different regions and backgrounds. Optimizing energy consumption is important, especially for deployment on edge devices where resources are limited. An offline mode can be introduced to ensure the system continues to function even without internet connectivity, improving reliability. Alert and notification features can be included to inform users about bin status, overflow conditions, or unusual patterns in waste collection. Finally, integrating the system with municipal or government waste management platforms can improve coordination, data sharing, and overall efficiency, contributing to a more organized and effective waste management process.

REFERENCES

1. Redmon, J., Divvala, S., Girshick, R., and Farhadi, A., “You Only Look Once: Unified, Real-Time Object Detection,” *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.
2. Bochkovskiy, A., Wang, C. Y., and Liao, H. Y. M., “YOLOv4: Optimal Speed and Accuracy of Object Detection,” *arXiv preprint arXiv:2004.10934*, 2020.
3. Jocher, G. et al., “YOLOv5 by Ultralytics,” *GitHub Repository*, 2020.
4. Goodfellow, I., Bengio, Y., and Courville, A., *Deep Learning*, MIT Press, 2016.
5. Krizhevsky, A., Sutskever, I., and Hinton, G., “ImageNet Classification with Deep Convolutional Neural Networks,” *Advances in Neural Information Processing Systems*, 2012.
6. Simonyan, K., and Zisserman, A., “Very Deep Convolutional Networks for Large-Scale Image Recognition,” *arXiv preprint arXiv:1409.1556*, 2014.
7. He, K., Zhang, X., Ren, S., and Sun, J., “Deep Residual Learning for Image Recognition,” *Proceedings of CVPR*, 2016.
8. Everingham, M. et al., “The Pascal Visual Object Classes (VOC) Challenge,” *International Journal of Computer Vision*, 2010.
9. Lin, T. Y. et al., “Microsoft COCO: Common Objects in Context,” *European Conference on Computer Vision (ECCV)*, 2014.
10. Chollet, F., *Deep Learning with Python*, Manning Publications, 2017.
11. Tan, M., and Le, Q., “EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks,” *ICML*, 2019.
12. Abadi, M. et al., “TensorFlow: Large-Scale Machine Learning on Heterogeneous Systems,” *Google Research*, 2015.

APPENDIX A

CODING

FRONTEND CODE:

```
from flask import Flask, render_template, Response, send_file
import sqlite3
import pandas as pd
import threading
import cv2
import time
import detector # use your working detector.py

from detector import start_detection, stop_detection

app = Flask(__name__)
thread = None

# ----- HOME PAGE -----
@app.route("/")
def home():
    return render_template("index.html", title="Home")

# ----- START CAMERA -----
@app.route("/start")
def start_cam():
    global thread
    if thread is None or not thread.is_alive():
        thread = threading.Thread(target=start_detection)
        thread.daemon = True
        thread.start()
    return render_template("live.html", title="Live Detection")
```

```

# ----- STOP CAMERA -----
@app.route("/stop")
def stop_cam():
    stop_detection()
    return {"status": "stopped"}

# ----- LIVE STREAM -----
@app.route("/video_feed")
def video_feed():

    def gen():
        while True:
            if detector.last_frame is not None:
                ret, jpeg = cv2.imencode('.jpg', detector.last_frame)
                if ret:
                    yield (
                        b'--frame\r\n'
                        b'Content-Type: image/jpeg\r\n\r\n' +
                        jpeg.tobytes() +
                        b'\r\n'
                    )
                time.sleep(0.03)

    return Response(gen(), mimetype='multipart/x-mixed-replace; boundary=frame')

# ----- HISTORY PAGE -----
@app.route("/history")
def history():
    try:
        conn = sqlite3.connect("history.db")
        df = pd.read_sql("SELECT * FROM data", conn)
        conn.close()
    except:
        df = pd.DataFrame(columns=["id", "material", "confidence", "timestamp"])

    return render_template("history.html",
                           title="History",
                           table=df.to_html(index=False))

```

```

# ----- EXPORT TO EXCEL -----
@app.route("/export")
def export():
    try:
        conn = sqlite3.connect("history.db")
        df = pd.read_sql("SELECT * FROM data", conn)
        conn.close()
    except:
        df = pd.DataFrame(columns=["id", "material", "confidence", "timestamp"])

    filename = "history.xlsx"
    df.to_excel(filename, index=False)

    return send_file(filename, as_attachment=True)

# ----- RUN APP -----
if __name__ == "__main__":
    app.run(debug=False, threaded=True, use_reloader=False)

```


BACKEND CODE

```
import cv2
import numpy as np
import tensorflow as tf
import pandas as pd
import os
import serial
import sqlite3
import time

running = False
last_frame = None
prev_gray = None

# ----- SERIAL SETUP -----
arduino = serial.Serial(port='COM6', baudrate=115200,
timeout=0.1)
print("Modules Loaded")

# ----- DATASET PREPROCESS -----
def preprocess(sdir, trsplit, vsplit):
    filepaths = []
    labels = []

    for klass in os.listdir(sdir):
        class_dir = os.path.join(sdir, klass)
        if os.path.isdir(class_dir):
            for f in os.listdir(class_dir):
                filepaths.append(os.path.join(class_dir, f))
                labels.append(klass)
```

```

df = pd.DataFrame({'filepaths': filepaths, 'labels': labels})

from sklearn.model_selection import train_test_split

dsplit = vsplit / (1 - trsplit)

train_df, dummy_df = train_test_split(
    df,
    train_size=trsplit,
    shuffle=True,
    random_state=1,
    stratify=df['labels']
)

valid_df, test_df = train_test_split(
    dummy_df,
    train_size=dsplit,
    shuffle=True,
    random_state=1,
    stratify=dummy_df['labels']
)

return train_df, valid_df, test_df

sdir = "labels"
img_size = (200, 200)

train_df, valid_df, test_df = preprocess(sdir, 0.90, 0.05)

```

```
from keras.preprocessing.image import ImageDataGenerator
```

```
train_gen =  
ImageDataGenerator(horizontal_flip=True).flow_from_data  
frame(  
    train_df,  
    x_col='filepaths',  
    y_col='labels',  
    target_size=img_size,  
    class_mode='categorical',  
    batch_size=20  
)
```

```
classes = list(train_gen.class_indices.keys())
```

```
model = tf.keras.models.load_model("model.h5")  
print("Model Loaded Successfully")
```

```
# ----- SAVE TO DATABASE -----
```

```
def save_history(material, confidence):
```

```
    try:
```

```
        conn = sqlite3.connect("history.db")
```

```
        conn.execute("""
```

```
        CREATE TABLE IF NOT EXISTS data(  
            id INTEGER PRIMARY KEY AUTOINCREMENT,
```

```
            material TEXT,
```

```
            confidence REAL,
```

```
            timestamp TEXT)
```

```

        conn.execute(
            "INSERT INTO data(material, confidence,
timestamp) VALUES (?, ?, datetime('now'))",
            (material, confidence)
        )

```

```

        conn.commit()

```

```

        conn.close()

```

```

except Exception as e:

```

```

    print("DB ERROR:", e)

```

```

# ----- MOTION DETECTION -----

```

```

def object_moving(frame):

```

```

    global prev_gray

```

```

    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

```

```

    gray = cv2.GaussianBlur(gray, (21, 21), 0)

```

```

    if prev_gray is None:

```

```

        prev_gray = gray

```

```

        return False

```

```

    frame_delta = cv2.absdiff(prev_gray, gray)

```

```

    thresh = cv2.threshold(frame_delta, 25, 255,
cv2.THRESH_BINARY)[1]

```

```

    thresh = cv2.dilate(thresh, None, iterations=2)

```

```
contours, _ = cv2.findContours(thresh,  
cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
```

```
prev_gray = gray
```

```
for c in contours:
```

```
    if cv2.contourArea(c) > 5000:
```

```
        return True
```

```
return False
```

```
CONF_THRESHOLD = 50.0
```

```
# ----- CAMERA LOOP -----
```

```
def start_detection():
```

```
    global running, last_frame
```

```
    running = True
```

```
    cap = cv2.VideoCapture(1, cv2.CAP_DSHOW)
```

```
    if not cap.isOpened():
```

```
        print("Camera not opening!")
```

```
        return
```

```
    plastic_count = 0
```

```
    metal_count = 0
```

```
    cardboard_count = 0
```

```

while running:
    ok, frame = cap.read()

    if not ok:
        continue

    try:
        # ----- CHECK IF OBJECT MOVING -----
        if not object_moving(frame):

            plastic_count = metal_count = cardboard_count =
0
            display_text = "WAITING FOR OBJECT"

            cv2.putText(frame, display_text, (30, 40),
                        cv2.FONT_HERSHEY_SIMPLEX, 1.2,
(0, 255, 255), 3)

            last_frame = frame.copy()
            continue

        # ----- PREPROCESS -----
        img = cv2.resize(frame, img_size)
        img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
        img = np.expand_dims(img, axis=0)

        pred = model.predict(img, verbose=0)

        index = np.argmax(pred)
        klass = classes[index]
        confidence = float(pred[0][index])

```

```

# ----- LOW CONFIDENCE -----
if confidence < CONF_THRESHOLD:

    plastic_count = metal_count = cardboard_count =
0

    display_text      =      f"Low      Confidence
({confidence:.1f}%)

    cv2.putText(frame, display_text, (30, 40),
                  cv2.FONT_HERSHEY_SIMPLEX, 1.2,
(0, 0, 255), 3)

    last_frame = frame.copy()
    continue

# ----- METAL -----
if "metal" in klass.lower():

    metal_count += 1
    plastic_count = cardboard_count = 0

    display_text      =      f"{klass.upper()}
{confidence:.1f}%"

    if metal_count > 4:
        display_text      =      f"FINAL      METAL
({confidence:.1f}%"
        arduino.write(b'A')
        time.sleep(2)
        save_history("METAL", confidence)
        metal_count = 0

```

```

# ----- PLASTIC -----
elif "plastic" in klass.lower():

    plastic_count += 1
    metal_count = cardboard_count = 0

    display_text = f'{klass.upper()}
{confidence:.1f}%'

    if plastic_count > 4:
        display_text = f'FINAL PLASTIC
({confidence:.1f}%)'
        arduino.write(b'B')
        time.sleep(2)
        save_history("PLASTIC", confidence)
        plastic_count = 0

# ----- CARDBOARD -----
elif "cardboard" in klass.lower():

    cardboard_count += 1
    plastic_count = metal_count = 0

    display_text = f'{klass.upper()}
{confidence:.1f}%'

    if cardboard_count > 8:
        display_text = f'FINAL CARDBOARD
({confidence:.1f}%)'
        arduino.write(b'C')
        time.sleep(2)

```



```

        save_history("CARDBOARD", confidence)
        cardboard_count = 0

    cv2.putText(frame, display_text, (30, 40),
                 cv2.FONT_HERSHEY_SIMPLEX, 1.2, (0,
255, 255), 3)

    last_frame = frame.copy()

except Exception as e:
    print("Detection Error:", e)
    last_frame = frame.copy()

cap.release()
print("Camera Released")

def stop_detection():
    global running
    running = False

```

ARDUINO CODE

```
#include <Servo.h>

#include <Wire.h>

#include <Adafruit_GFX.h>

#include <Adafruit_SSD1306.h>

#include <avr/wdt.h> // Include watchdog timer library

#define SCREEN_WIDTH 128

#define SCREEN_HEIGHT 64

#define OLED_RESET -1

Adafruit_SSD1306 display(SCREEN_WIDTH,
SCREEN_HEIGHT, &Wire, OLED_RESET);

#define IR_SENSOR_PIN 2 // IR sensor pin

#define CONTINUOUS_PIN 3 // Always running pin

#define SERVO1_PIN 5

#define SERVO2_PIN 6

#define SERVO3_PIN 9

Servo servo1, servo2, servo3;

bool irDetected = false;

void setup() {

Serial.begin(9600);

pinMode(IR_SENSOR_PIN, INPUT);

pinMode(CONTINUOUS_PIN, OUTPUT);

servo1.attach(SERVO1_PIN);

servo2.attach(SERVO2_PIN);

servo3.attach(SERVO3_PIN);
```

```

servo1.write(0);

servo2.write(90);

servo3.write(90);

if (!display.begin(SSD1306_SWITCHCAPVCC, 0x3C)) {
  Serial.println(F("SSD1306 allocation failed"));
  for (;;)
  }

display.clearDisplay();
display.setTextSize(1);
display.setTextColor(WHITE);
display.setCursor(0, 0);
display.println("System Ready...");
display.display();
wdt_disable();
delay(3000);
}

void loop() {
  int irValue = digitalRead(IR_SENSOR_PIN);
  if (irValue == LOW) { // IR Sensor Detected
    digitalWrite(CONTINUOUS_PIN, LOW); // Stop pin 3
    displayStatus("IR DETECTED: STOP");
    if (Serial.available()) {
      char command = Serial.read();
      Serial.println(command);
    }
  }
}

```

```

if (command == 'A') {

digitalWrite(CONTINUOUS_PIN, HIGH); // Run pin 3
delay(600);
digitalWrite(CONTINUOUS_PIN, LOW); // Run pin 3
moveServo(servo1, "Servo1 Running", 3);
servo1.write(90);
} else if (command == 'B') {
servo1.write(90);
delay(1000);
servo2.write(0);
digitalWrite(CONTINUOUS_PIN, HIGH); // Run pin 3
delay(2000);
digitalWrite(CONTINUOUS_PIN, LOW); // Run pin 3
moveServo(servo2, "Servo2 Running", 6);
} else if (command == 'C') {
servo1.write(90); delay(1000);
servo3.write(0);
digitalWrite(CONTINUOUS_PIN, HIGH); // Run pin 3
delay(3250);
digitalWrite(CONTINUOUS_PIN, LOW); // Run pin 3
moveServo(servo3, "Servo3 Running", 3);
}
}
}

```

```

Else {

digitalWrite(CONTINUOUS_PIN, HIGH); // Run pin 3

displayStatus("IR NOT DETECTED: RUNNING");

}

}

void moveServo(Servo &servo, const char *status, int d) {

displayStatus(status);

for (int pos = 0; pos <= 130; pos++) {

servo.write(pos);

delay(d);

}

for (int pos = 130; pos >= 0; pos--) {

servo.write(pos);

delay(d);

wdt_enable(WDTO_15MS); // Enable watchdog with 15ms
timeout

}

}

void displayStatus(const char *message) {

display.clearDisplay();

display.setCursor(0, 0);

display.println(message);

display.display();

}

```

APPENDIX B

PATENT DISCLOSURE FORM

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY		SRM INSTITUTE OF SCIENCE AND TECHNOLOGY KATTANKULATHUR – 603203																	
Application for patent filing																			
		Date	<table border="1" style="display: inline-table; text-align: center;"> <tr> <th>D</th><th>D</th><th>M</th><th>M</th><th>Y</th><th>Y</th><th>Y</th><th>Y</th> </tr> <tr> <td>2</td><td>0</td><td>0</td><td>4</td><td>2</td><td>0</td><td>2</td><td>6</td> </tr> </table>	D	D	M	M	Y	Y	Y	Y	2	0	0	4	2	0	2	6
D	D	M	M	Y	Y	Y	Y												
2	0	0	4	2	0	2	6												
Name of the Faculty	:	Dr. Jackulin Mahariba A																	
Department	:	CINTEL																	
Faculty ID Number	:	101370																	
Official E-mail ID	:	jackulia@srmist.edu.in																	
Contact no. of all Inventors	:	9841088361,																	
Major area of invention	:	Artificial Intelligence based Automation System for Waste Management																	
Narrow focus area of invention	:	Real-time waste detection, classification and automated mechanical segregation using YOLOv8 and embedded hardware control																	
Title of the invention	:	Automated Waste Sorting and Recycling System Using YOLO-Based Object Detection and Django-Integrated Monitoring																	
Earlier status of research	:	Existing systems for waste segregation rely on manual sorting or partial automation using conventional machine vision and sensor-based methods. Many reported works focus either on software-based classification or hardware-based sorting, but lack integrated real-time monitoring, database logging, and web-based analytics. Several deep learning approaches such as CNN and YOLO variants have been explored for classification, but hardware-software synchronized segregation with live tracking remains limited.																	

SRMIST - Confidential

1



underrepresented classes such as paper and plastic, to enhance classification accuracy. Model fine-tuning, expanded dataset training, and system performance optimization are planned as next development steps

12. Please list any of your publications (including abstracts, posters, news releases, etc.) to emphasize the present invention background.

At present, the work has been documented as an academic project report. A research manuscript based on this invention is being prepared; however, it has not yet been submitted or published in any journal or conference.

13. INVENTOR(S) AND/OR CONTRIBUTOR(S):

	INVENTOR (1)	INVENTOR (2)	INVENTOR (3)
Signature:	<i>H. Kishore</i>	<i>S. Koushik</i>	<i>Jackulin</i>
Name:	Kishore Khannan H	Koushik Vishal S	Jackulin Mahariba A
Address:	Department Computational Intelligence, SRM IST, Kattankulathur -603203	Department Computational Intelligence, SRM IST, Kattankulathur -603203	Department Computational Intelligence, SRM IST, Kattankulathur -603203
City and State:	Chennai & Tamilnadu	Chennai & Tamilnadu	Chennai & Tamilnadu
Citizenship	INDIAN	INDIAN	INDIAN
(Country):	INDIA	INDIA	INDIA

14. ASSIGNMENT DETAILS: Assignee is the entity or individual who holds the patent.

Signature: (To be signed by the authorized signatory on behalf of the assignee)

Name of the Authorized Signatory and Designation

Address:

City and State:

APPENDIX C

PLAGARISM REPORT



Page 2 of 46 - Integrity Overview

Submission ID trn:oid::1:3557358241

5% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Filtered from the Report

- Bibliography
- Quoted Text

Match Groups

-  **35 Not Cited or Quoted 5%**
Matches with neither in-text citation nor quotation marks
-  **0 Missing Quotations 0%**
Matches that are still very similar to source material
-  **0 Missing Citation 0%**
Matches that have quotation marks, but no in-text citation
-  **0 Cited and Quoted 0%**
Matches with in-text citation present, but no quotation marks

Top Sources

- 0%  Internet sources
- 2%  Publications
- 3%  Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

No suspicious text manipulations found.

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.



Page 2 of 46 - Integrity Overview

Submission ID trn:oid::1:3557358241

